Nombres: Sara Ortiz y Roxana Soto
Repositorio GitHub: https://github.com/Roxana1492/Proyecto-2-DPOO

Contexto y alcance

1. **Descripción del problema**
   Un club deportivo administra un sistema que abarca la gestión de disciplinas deportivas (deportes individuales y grupales, torneos, reservas de instalaciones y rankings) y también el área administrativa y comercial (inventarios, tienda, cafetería, restricción de alérgenos y turnos de personal).
   El club ya contaba con el sistema del Proyecto 1, que maneja la parte deportiva (equipos, inscripciones, partidos, rankings, reservas y entrenamientos) y la parte comercial (tienda, cafetería, inventario y turnos). Como la asistencia a los partidos internos bajó, la administración decidió organizar torneos internos para que los socios participen más. Por eso en este proyecto se agregó un componente nuevo de torneos y se cambió la forma en que el usuario interactúa con el sistema.
   Los torneos pueden ser recreativos o competitivos. Los recreativos no cobran inscripción y premian con un bono de descuento para la tienda o la cafetería. Los competitivos cobran una tarifa y reparten un premio en efectivo según lo recaudado. Un torneo puede ser de un deporte de conjunto, donde se inscriben equipos completos, o de una disciplina individual, donde se inscriben jugadores o parejas si es de dobles. Además de los socios, ahora pueden participar estudiantes de la universidad y personas externas, y los empleados pueden inscribirse a torneos individuales si no tienen turno ese día.
   También cambió la interfaz. En el Proyecto 1 había un solo programa de consola con todos los menús en la clase CoordinadorAplicacion. En este proyecto hay tres programas, uno por cada tipo de usuario principal (administrador, participante y empleado), todos validan las entradas y cargan y guardan los datos en archivos fijos sin pedirle nada al usuario.

2. **Requerimientos funcionales, Requerimientos no funcionales y restricciones**

   Requerimientos funcionales

   > Los requerimientos funcionales se reescribieron en una forma más sencilla a comparación de la entrega 1.

   RF1: Inscribir a un socio en un equipo deportivo grupal validando su edad, cupo, número de camiseta único y que esté inscrito solo en un equipo por deporte.
   RF2: Inscribir un socio en un deporte individual asignándole un nivel (principiante, intermedio o avanzado).
   RF3: Reservar una cancha o mesa para una disciplina validando el cupo máximo de instalaciones simultáneas y la restricción de no entrar bebidas calientes.

RF4: Programar un partido de práctica entre jugadores del mismo nivel o adyacente, o con un entrenador.
RF5: Registrar las estadísticas de un partido de práctica y actualizar el ranking de los jugadores.
RF6: Consultar el historial de estadísticas de partidos y la evolución del ranking en cada deporte.
RF7: Consultar el historial de asistencia y las notas o comentarios de las evaluaciones de desempeño por sesión.
RF8: Consultar los resultados de partidos, tabla de posiciones de torneos, fases y clasificaciones de goleadores o jugadores destacados.
RF9: Comprar artículos deportivos en una sola transacción calculando el IVA y generando el 2% en puntos.
RF10: Comprar bebidas o snacks indicando alérgenos, calculando impuestos al consumo y modificando la propina.
RF11: Rechazar la venta de una bebida caliente a un socio que se encuentra dentro de una cancha de pádel o una mesa de tenis de mesa techada.
RF12: Aplicar códigos de descuento de empleados (15% si es propio o 8% si es compartido).
RF13: Solicitar un cambio de turno y consultar el estado de solicitud (aprobada, rechazada o pendiente).
RF14: Planear sesiones de entrenamiento indicando fecha, hora, duración, instalación y jugadores convocados.
RF15: Registrar la asistencia y evaluación opcional de cada jugador al finalizar una sesión de entrenamiento.
RF16: Registrar el resultado y las estadísticas de un partido, validando antes que ambos equipos tengan el mínimo de jugadores habilitados.
RF17: Registrar tarjetas amarillas y rojas en fútbol y marcar como suspendido al jugador que acumule 2 amarillas o 1 roja.
RF18: Sugerir nuevos artículos o snacks por parte de los empleados para que el administrador evalúe incorporarlos.
RF19: Consultar el estado del inventario de artículos por ubicación y la disponibilidad de instalaciones.
RF20: Consultar el historial de reservas, partidos y sesiones de entrenamiento del club.
RF21: Crear y dar de baja equipos y jugadores de forma actualizada sin perder su historial de estadísticas.
RF22: Mover artículos entre ubicaciones y reabastecer el inventario de la tienda.
RF23: Aprobar o rechazar las sugerencias de nuevos artículos hechas por los empleados para añadirlas al catálogo.
RF24: Asignar los turnos semanales de entrenadores y fisioterapeutas asegurando la cobertura mínima del personal.
RF25: Aprobar o rechazar solicitudes de cambio de turno garantizando que no se incumpla la regla de la cobertura mínima del personal.
RF26: Generar reportes financieros de ventas filtrados por categoría (tienda y cafetería) y tiempo.

RF27: Generar reportes deportivos independientes de asistencia, resultados de partidos y evolución de rankings.
RF28: Iniciar sesión con usuario y contraseña según el rol del usuario.
RF29: Inscribir un equipo en una competencia, validando su categoría y la nómina mínima de jugadores habilitados.
RF30: Registrar un partido de competencia, permitiendo alinear únicamente a jugadores registrados en la nómina.
RF31: Calcular la tabla de posiciones según las reglas de puntos del deporte y determinar el avance entre fases.
RF32: Rechazar automáticamente la alineación de un jugador que se encuentre suspendido por acumulación de tarjetas.
Los requerimientos RF33 a RF42 son nuevos del Proyecto 2 y salen de las historias de usuario de torneos (HU-T01 a HU-T13).
RF33: Crear un torneo recreativo o competitivo definiendo deporte o disciplina, modalidad (conjunto, individual o dobles), categoría por edad máxima o nivel, fecha, número de cupos y tarifa si es competitivo.
RF34: Inscribir a un jugador, una pareja o un equipo completo en un torneo que tenga las inscripciones abiertas, validando la categoría y los cupos disponibles.
RF35: Salirse de un torneo antes de que inicie, liberando los cupos que se habían tomado y devolviendo la tarifa al fondo si se había pagado.
RF36: Reservar el 20% de los cupos de cada torneo, redondeado hacia arriba, para los jugadores o equipos con mejor ranking de la categoría, y liberarlos como cupos regulares si no se llenan al iniciar el torneo.
RF37: Permitir que entrenadores y fisioterapeutas se inscriban a torneos individuales o de dobles si no tienen turno el día del torneo, sin pagar inscripción en los competitivos y sin recibir premio en efectivo.
RF38: Registrar la cuenta de un estudiante matriculado en la universidad, validando su código estudiantil, o de una persona externa.
RF39: Iniciar un torneo, generando el calendario de encuentros todos contra todos, registrar el resultado de cada encuentro y finalizar el torneo cuando todos los encuentros tengan resultado.
RF40: Entregar los premios al finalizar el torneo: un bono de descuento a los ganadores de un recreativo y el fondo recaudado repartido 60%, 30% y 10% entre los tres primeros de un competitivo.
RF41: Guardar el bono ganado en el perfil del usuario y aplicarlo en una compra futura de la tienda o la cafetería, sin acumularlo con puntos ni con códigos de descuento de empleado.
RF42: Consultar el estado de cualquier torneo (inscripciones abiertas, en curso o finalizado) y su historial detallado: participantes, encuentros, posiciones y premios.

Requerimientos no funcionales

> Se agruparon los requerimientos funcionales por grupos generales para poder hacer los requerimientos no funcionales por secciones y aclarar el uso de la consola.

RNF1 - Facilidad de cambios

Agregar un deporte nuevo o una nueva regla de puntuación al sistema debe implicar la creación de clases nuevas, pero sin añadir modificaciones en el código de los módulos de torneos, los rankings, ni el registro de estadísticas. Se verifica agregando un deporte de prueba y contando cuántas clases existentes hubo que modificar.

RNF2 - Tolerancia a fallos y consistencia de datos

Cuando el sistema rechace una operación por incumplir una regla del club, el estado interno del sistema debe quedar exactamente igual a como estaba antes del intento, tanto en la memoria como en los archivos. Además, en procesos largos como una compra de varios artículos, si el último producto falla, los anteriores no pueden haber descontado inventario.

RNF3 - Aislamiento de la persistencia

La lectura de catálogos y almacenamiento local deben estar totalmente separados de las reglas del negocio. La regla clave es que un cambio en el formato de los archivos debe afectar únicamente a los componentes de persistencia sin alterar el resto de la aplicación.

RNF4 - Parámetros financieros

Al modificar el valor del IVA, impuesto al consumo, porcentaje de puntos o de los descuentos, se debe requerir cambiar en un único lugar sin cambiar el flujo de las transacciones. Se verifica cambiando el valor del IVA y comprobando que solo se editó una parte.

RNF5 - Consola

La consola del sistema debe mostrar la información de manera clara y ordenada, confirmando el resultado de cada acción, ya sea exitosa o rechazada con su respectivo resultado o razón.

RNF6 - Validación de entradas en la consola

Ninguna entrada del usuario puede hacer que el programa se cierre con un error. Si el usuario escribe un número fuera de rango, una fecha con formato incorrecto o deja un campo vacío, la consola vuelve a pedir el dato explicando qué estuvo mal. La validación de formato no debe repetirse en cada menú, sino estar en una sola clase que usen las tres consolas.

RNF7 - Persistencia automática

Al iniciar cualquiera de los tres programas se cargan los datos desde archivos predeterminados de la carpeta datos/, y al cerrar se guardan los cambios en esos mismos archivos. El usuario nunca escribe el nombre de un archivo. No es necesario guardar después de cada acción, basta con hacerlo al terminar el programa.

RNF8 - Pruebas automáticas

La lógica de torneos y las clases que se modificaron deben tener pruebas unitarias y de integración con JUnit, con al menos una prueba de integración por cada requerimiento funcional nuevo. La consola no se prueba con JUnit.

Restricciones

Las restricciones se copiaron del documento de la entrega 1. Las últimas nueve son nuevas del Proyecto 2 y salen del enunciado de los torneos.

- ➔ Un jugador solo puede pertenecer a un equipo por deporte, aunque sí puede jugar varios deportes del conjunto a la vez.
- ➔ La categoría de un equipo debe corresponder a la edad del jugador inscrito.
- ➔ El número de la camiseta debe ser único dentro de cada equipo.
- ➔ Un partido interno solo puede jugarse si ambos equipos cumplen con el mínimo de jugadores requeridos para cada deporte.
- ➔ Un jugador con 2 tarjetas amarillas o una roja queda suspendido para el siguiente partido del equipo.
- ➔ Solo los jugadores en un equipo de una competencia pueden disputar sus partidos oficiales.
- ➔ La asignación de puntos a los equipos de fútbol es 3, 1 o 0 dependiendo si ganan, empatan o pierden, respectivamente. En baloncesto y voleibol no hay empates.
- ➔ Un partido de práctica individual sólo puede programarse entre niveles iguales o adyacentes, salvo que uno de los jugadores sea un entrenador.
- ➔ Las instalaciones tienen capacidad fija según el tipo y existe un máximo de instalaciones simultáneas por disciplina.
- ➔ No se puede entregar una bebida caliente a un socio dentro de una cancha de pádel o mesa de tenis techada, ni permitir el uso de esas instalaciones a alguien con una bebida caliente.
- ➔ Toda compra en la tienda paga IVA; toda venta en la cafetería paga impuesto al consumo y admite propina sugerida.
- ➔ El código de descuento de un empleado da 15% a sí mismo y 8% a otro socio con quien lo comparta.
- ➔ Cada venta genera 2% de su valor en puntos redimibles en compras.
- ➔ Debe haber siempre mínimo 2 entrenadores y 1 fisioterapeuta disponibles durante el horario de atención; las solicitudes de cambio de turno solo se aprueban si no incumplen la regla.
- ➔ El administrador no puede inscribirse a partidos ni reservar instalaciones para uso personal.

- ➔ Un torneo es recreativo o competitivo. El recreativo no tiene tarifa y el competitivo tiene una tarifa mayor que cero.
- ➔ Puede haber varios torneos al mismo tiempo, y cada uno se juega en una sola fecha.
- ➔ En los torneos de conjunto cada cupo es un equipo completo. En dobles cada pareja ocupa dos cupos, por eso el número de cupos debe ser par.
- ➔ El 20% de los cupos se reserva para los mejores del ranking. Si esos cupos no se llenan, se liberan como cupos regulares.
- ➔ Solo se puede inscribir o salir de un torneo mientras sus inscripciones estén abiertas.
- ➔ Los empleados no pueden inscribirse a torneos de conjunto ni a torneos que caigan en un día en que tengan turno.
- ➔ Un administrador no participa en torneos.
- ➔ El bono de descuento es del 20% sobre el subtotal, se usa una sola vez, solo lo puede usar su dueño y no se acumula con otros descuentos.
- ➔ Los pagos están tercerizados, el sistema solo guarda la confirmación del pago.

3. Objetivos y No objetivos

Objetivos

O1: Automatizar la gestión deportiva del club permitiendo la inscripción de socios en deportes grupales e individuales, torneos y ligas.
O2: Controlar la reserva de instalaciones y programación de partidos, actualizando estadísticas y rankings de manera automatizada.
O3: Planificar sesiones de entrenamiento registrando las convocatorias, asistencias y evaluaciones de desempeño por parte de los entrenadores.
O4: Administrar las operaciones de la tienda y la cafetería calculando impuestos, propinas e implementando restricciones de alérgenos y bebidas calientes.
O5: Administrar los recursos del club controlando inventarios, asignando turnos de entrenadores y fisioterapeutas y asegurando la cobertura mínima del personal.
O6: Organizar torneos internos recreativos y competitivos que aumenten la participación de socios, estudiantes y externos.
O7: Manejar los cupos de cada torneo de forma justa, dando prioridad a los jugadores con mejor ranking.
O8: Separar la interfaz de consola de la lógica del sistema, con una consola por tipo de usuario y validación de todas las entradas.

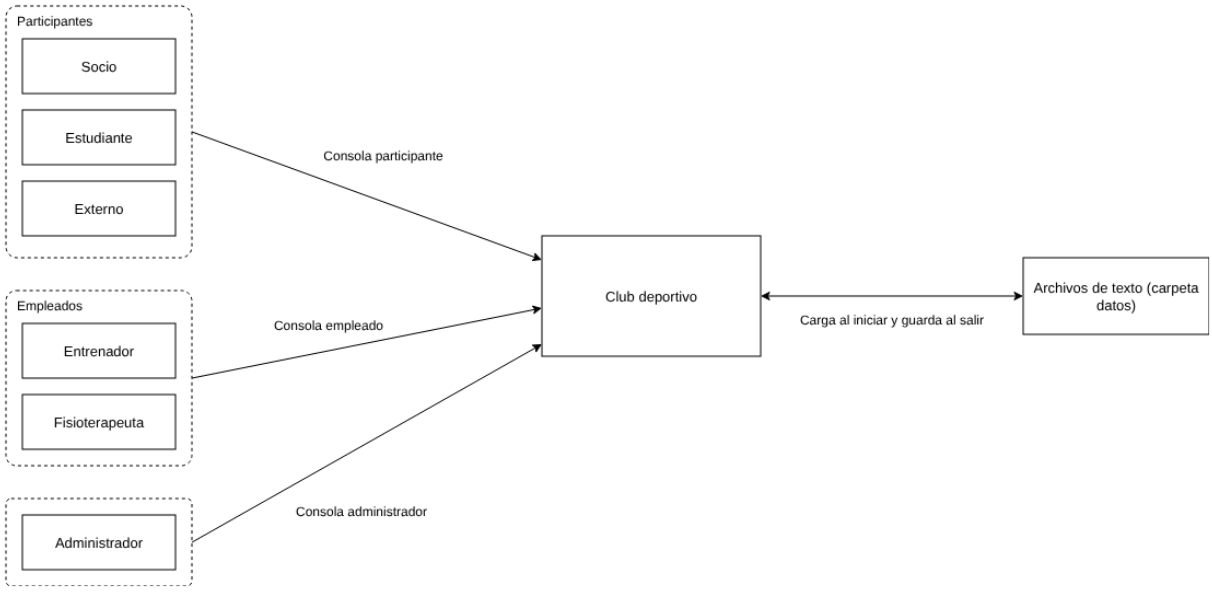
No objetivos

NO1: El uso del ranking para la selección automática de emparejamientos y las visualizaciones avanzadas de rendimiento.
NO2: La gestión contable de mensualidades o cuotas de afiliación monetarias de los socios.

NO3: La modelación detallada de equipos rivales de otras instituciones como entidades complejas con historial propio.
NO4: Interfaces gráficas de usuario o web, ya que el sistema opera de forma exclusiva por consola de texto.
NO5: Integración con pagos en línea o procesamiento de tarjeta de crédito.
NO6: Procesar pagos de inscripciones o pagar los premios en efectivo. El sistema solo registra cuánto se le debe a cada ganador.
NO7: Formatos de torneo distintos a la liga todos contra todos, como eliminación directa.
NO8: Que varias personas usen el sistema al mismo tiempo sobre los mismos archivos.

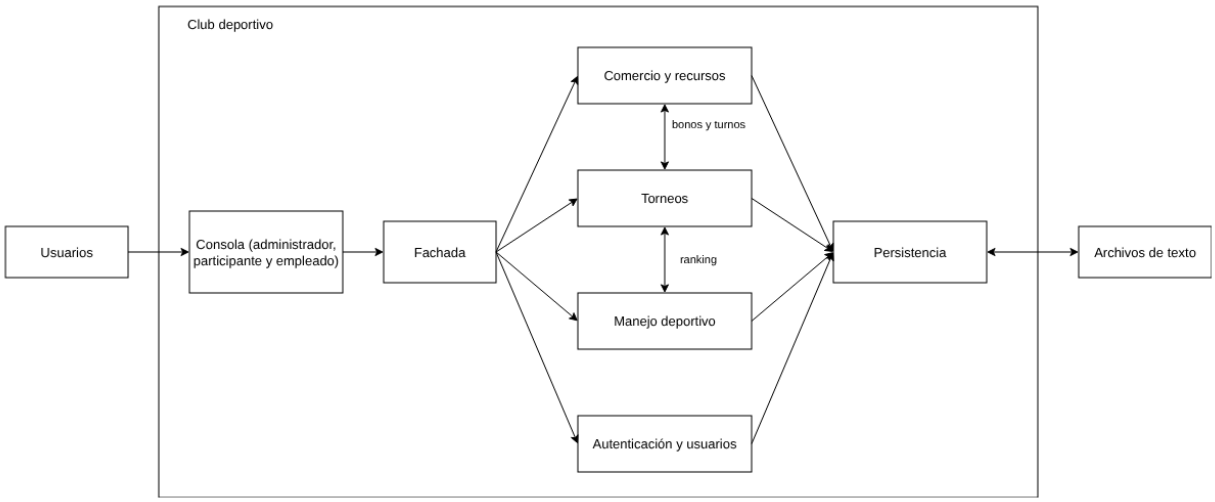
Descomponer componentes

Diagrama de contexto



El diagrama de contexto sigue mostrando el sistema del club como una caja central y los archivos de texto donde se guarda la información. Lo que cambió es quién usa el sistema y por dónde entra. Ahora hay tres grupos de usuarios y cada uno tiene su propia consola: los participantes (socios, estudiantes y externos) usan la consola de participante, los empleados (entrenadores y fisioterapeutas) usan la consola de empleado y el administrador usa la consola de administrador.
Los archivos ya no se cargan cuando el usuario lo pide, sino que el programa los lee al iniciar y los guarda al salir, por eso la flecha hacia los archivos de texto dice eso.

Diagrama de componentes



El diagrama de componentes está dividido en 7 componentes:

- Consola: tiene las tres aplicaciones (administrador, participante y empleado), la clase base que comparten y el lector de entradas que valida todo lo que escribe el usuario. Este componente no toma ninguna decisión del negocio, solo pide datos y muestra resultados.
- Fachada: es la única forma en que la consola llega a la lógica. Tiene la interfaz FachadaTorneos, la clase que la implementa y la clase OperacionesComercio, que tiene las operaciones de tienda, cafetería, turnos y reportes que antes estaban en CoordinadorAplicacion.
- Torneos: es el componente nuevo. Maneja la creación de torneos, las inscripciones, los cupos preferenciales, el calendario, los resultados, la tabla de posiciones y los premios.
- Autenticación y usuarios: se mantiene la jerarquía de usuarios del Proyecto 1 y se agregan los estudiantes y los externos, junto con el registro de matrículas que sirve para validar a los estudiantes.
- Manejo deportivo y Comercio y recursos: siguen igual que en el Proyecto 1. Torneos usa del primero el ranking, y de Comercio y recursos usa el registro de turnos para saber si un empleado puede jugar.
- Persistencia: además del almacenador de archivos del Proyecto 1, tiene una clase nueva que carga y guarda los usuarios, los torneos y los bonos.

Se decidió poner la fachada entre la consola y la lógica por dos razones. La primera es que así la consola no necesita conocer las clases internas de los torneos, solo los métodos de la interfaz, y si cambia la forma de guardar un torneo o de calcular los cupos la consola no se toca (RNF5). La segunda es práctica: como el trabajo se dividió entre las dos integrantes, la interfaz quedó como un acuerdo de qué necesita la consola y qué debe devolver la lógica, y cada una pudo avanzar en su parte sin esperar a la otra.

Torneos se dejó como un componente aparte y no dentro de Manejo deportivo porque sus reglas son diferentes a las de las competencias del Proyecto 1. Los torneos tienen cupos, tarifas, premios y participantes que no son socios, y mezclar eso con las ligas y fases que ya existían haría que un cambio en los premios pudiera afectar el cálculo de la tabla de las competencias.
La lógica deportiva y la comercial siguen separadas por la misma razón del Proyecto 1: que los cambios comerciales, como el IVA o las propinas, no afecten la lógica de los torneos o de los rankings.
Se separó el módulo de persistencia del resto de la aplicación porque se asegura que cualquier cambio en el formato de los archivos de texto cambie únicamente ese componente.
Finalmente, la independencia de la interfaz se hizo porque si la consola evoluciona o cambia a una interfaz gráfica, ni el módulo de lógica ni persistencia tendrán cambios.

Candidatos y estereotipos

> Las clases candidatas se van a organizar por componentes (los del diagrama de componentes): Autenticación y consola, Gestión deportiva, Comercio y recursos y Persistencia, las cuales están señaladas por colores, azul, verde, morado y naranja, respectivamente. En el Proyecto 2 se quitaron Consola menú y Coordinador aplicación, y las clases nuevas o modificadas (consola, fachada, torneos, usuarios y persistencia) se agregaron al final de la tabla en amarillo.

En esta sección se definen las clases candidatas que conforman la parte interna del sistema, asegurando cubrir los requerimientos funcionales y no funcionales.

Candidato	Descripción	Estereotipo
Autenticador	Valida el usuario y la contraseña al ingresar y registra el rol correspondiente.	Controller
Usuario	Almacena los atributos base y datos de identificación de todas las personas registradas.	Information holder
Socio	Almacena el perfil del socio, incluyendo antigüedad, contacto y membresía.	Information holder
Entrenador	Almacena la hoja de vida, deportes a cargo y horario de trabajo de cada uno.	Information holder
Fisioterapeuta	Almacena los datos del personal de salud encargado de las evaluaciones y tratamientos, y el horario de trabajo de cada uno.	Information holder
Administrador	Almacena el perfil operativo con los permisos extendidos de gestión general.	Information holder
Deporte	Representa una disciplina definiendo sus reglas de puntuación, alineación e indicadores.	Information holder
Estadísticas	Almacena los indicadores de cada deporte específico (goles, puntos o faltas).	Information holder
Ranking	Calcula y actualiza el puntaje de los competidores y los equipos según los resultados.	Service provider
Sanción deportiva	Almacena y cuenta la acumulación de tarjetas, suspensiones y penalizaciones	Information holder

	aplicadas a un jugador.	
Verificación sanción	Verifica si un deportista tiene registrado suspensiones o tarjetas acumuladas lo que le impide participar en el siguiente encuentro deportivo.	Controller
Competencia	Organiza la estructura, fases y equipos que participan en un torneo o liga.	Structurer
Equipo	Agrupa los socios que son parte de un deporte grupal y están habilitados para competir.	Structurer
Partido	Registra la fecha, espacio, equipos y al final del partido el marcador.	Structurer
Tabla de posiciones	Calcula y ordena los puntajes y rendimiento de los equipos en una competencia.	Service provider
Inscripción equipo	Registra y almacena el encuentro, fecha y estado de un equipo dentro de una competencia.	Information holder
Inscripción jugador	Registra y almacena a un socio dentro de un equipo, asignando el número de su camiseta y posición dentro del deporte.	Information holder
Verificación inscripción	Verifica las reglas de aceptación a equipos y cupos antes de hacer la inscripción.	Controller
Verificación alineación	Verifica que los socios no tengas sanciones o tarjetas antes de los partidos.	Controller
Sesión de entrenamiento	Registra la asistencia y evaluación de los deportistas en una práctica.	Information holder
Evaluación de rendimiento	Registra los datos de rendimiento físico evaluados de un deportista durante un entrenamiento	Information holder
Reserva de instalación	Administra la asignación de canchas o mesas deportivas, evitando los cruces de horario.	Controller
Verificación reserva	Verifica la disponibilidad de la instalación y valida que el socio no exceda el límite de reservas.	Controller

Producto	Guarda el nombre, código, precio y cantidad del producto en la tienda o cafetería.	Information holder
Alimento cafetería	Almacena los productos de cafetería poniendo explícitamente el contenido de alérgenos.	Information holder
Sugerencia producto	Registra las solicitudes de nuevos productos por los empleados.	Information holder
Inventario	Verifica las existencias de mercancía y actualiza las cantidades disponibles.	Structurer
Venta	Registra los datos de una transacción realizada en la tienda o la cafetería.	Information holder
Cuenta puntos	Almacena el saldo actual de puntos acumulados de fidelización de un socio.	Information holder
Calculador puntos	Calcula la cantidad de puntos ganados por cada compra o la conversión de puntos a dinero.	Service provider
Gestor descuentos	Procesa el descuento que se quiere aplicar, dependiendo del usuario que lo gestione.	Service provider
Parámetro financiero	Almacena los valores del IVA, impuesto al consumo, tasa de puntos y descuentos.	Information holder
Regla de bebida caliente	Verifica que no haya bebidas calientes para consumo en cancha de pádel y tenis de mesa techado.	Controller
Impuestos	Calcula las tasas de IVA o propinas según el producto.	Service provider
Turnos	Almacena la asignación de turnos de horario, rol y empleado responsable.	Information holder
Solicitud cambio turnos	Almacena el estado (pendiente, aprobado o rechazado), motivos y empleados que solicitan el cambio de turno	Information holder
Registro turnos	Organiza los horarios y turnos de trabajo del club, verificando que haya la cobertura de personal mínima.	Controller
Generador reportes	Procesa y une los datos de las ventas, reservas y asistencias para crear los	Service provider

	informes administrativos.	
Almacenador de archivos	Lee y escribe la información del sistema en los archivos de texto.	Interfacer
Consola base	Tiene lo que comparten las tres consolas: inicio de sesión, ciclo del menú, manejo de errores y guardado al salir.	Coordinator
Lector de entradas	Lee lo que escribe el usuario y lo vuelve a pedir hasta que tenga el formato correcto.	Interfacer
Consola administrador	Muestra el menú del administrador para crear, iniciar y finalizar torneos, registrar resultados y ver reportes.	Interfacer
Consola participante	Muestra el menú de socios, estudiantes y externos para registrarse, inscribirse, ver bonos y comprar.	Interfacer
Consola empleado	Muestra el menú de entrenadores y fisioterapeutas para inscribirse a torneos y pedir cambios de turno.	Interfacer
Fachada torneos	Define las operaciones que la consola le puede pedir a la lógica.	Interfacer
Club fachada	Recibe las peticiones de la consola y las reparte entre el gestor de torneos, el autenticador y la persistencia.	Controller
Operaciones comercio	Hace las compras en tienda y cafetería, los cambios de turno y los reportes, sin leer ni imprimir nada.	Service provider
Gestor torneos	Crea y guarda los torneos y decide si un participante entra por cupo preferencial o regular.	Controller
Torneo	Guarda los datos del torneo, sus cupos y su estado, y controla las inscripciones, los resultados y el cierre.	Information holder
Torneo recreativo	Torneo sin tarifa que premia con un bono de descuento.	Information holder
Torneo competitivo	Torneo con tarifa que guarda el fondo recaudado y lo reparte en efectivo.	Information holder
Categoría	Guarda la edad máxima o el nivel que se exige para entrar al torneo.	Information holder

Inscripción torneo	Guarda quién se inscribió, si tomó un cupo preferencial y cuánto pagó.	Information holder
Participante torneo	Representa lo que se inscribe en un torneo, sin importar si es un jugador, una pareja o un equipo.	Interfacer
Jugador, Pareja y Equipo torneo	Son las tres formas de participar en un torneo. El equipo además conoce a su capitán.	Information holder / Structurer
Encuentro	Guarda los dos participantes de un partido del torneo, la ronda y el marcador.	Information holder
Generador calendario	Arma las rondas de la liga todos contra todos, con descansos si el número es impar.	Service provider
Tabla torneo y Posición torneo	Llevan los puntos, marcadores y el orden de los participantes aplicando el desempate.	Structurer / Information holder
Premio, Premio efectivo y Bono descuento	Guardan lo que gana cada participante. El bono además sabe si ya se usó y quién es su dueño.	Information holder
Estudiante y Externo	Nuevos tipos de usuario con nombre, documento, fecha de nacimiento y nivel. El estudiante tiene código estudiantil.	Information holder
Registro matrículas	Guarda los códigos de los estudiantes matriculados para validar el registro.	Information holder
Persistencia club	Carga y guarda los usuarios, torneos, inscripciones, encuentros, premios y bonos en la carpeta datos/.	Service provider

Definición de los tipos de estereotipos utilizados

- Information holder - Posee información, almacena, mantiene y entrega datos sobre las mismas clases.
- Structurer - Son clases que agrupan, contienen u organizan los objetos y conocen las relaciones estructurales entre ellos.
- Service provider - Clases que realizan un cálculo o transformación de los datos cuando se solicitan.
- Controller - Clases que toman decisiones aplicando reglas de negocio, validan estados y coordinan acciones para controlar.
- Coordinator - Son clases que distribuyen el flujo de trabajo entre otros componentes sin asumir la lógica de las decisiones.

- Interfacer - Clases que comunican los factores externos con la lógica central del sistema.

Clases descartadas

Durante el análisis se tuvieron en cuenta ciertas clases candidatas que al final no se pusieron por las siguientes razones

- Categoría de edad: se descarta porque es una propiedad descriptiva dentro del equipo y competencia, no requiere un comportamiento propio.
- Número de camiseta: Se modela como un atributo numérico dentro de la clase de inscripción jugador, acompañados de una regla de validación, en lugar de una clase separada.
- Puntos de posiciones: No es información propia de un partido. Se calculan dentro de la clase tabla de posiciones a partir del resultado de cada partido y los puntos de cada deporte. No se mezclan con los puntos de fidelidad de los socios ya que estos sí tienen su propia clase.
- Estado turno: Son estados de cambio de turno (aprobado, pendiente o rechazado) dentro de la clase turno.
- Precio: es un atributo numérico dentro de la clase de producto dejando sus operaciones de cálculo dentro del módulo de comercio.

Asignar responsabilidades

En esta sección se definen las responsabilidades del sistema, garantizando que cada responsabilidad se asigna a una sola clase. Las responsabilidades de las clases del Proyecto 1 se mantienen, con una excepción: las que tenían CoordinadorAplicacion (navegar entre menús y hacer las compras, ventas, turnos y reportes) y ConsolaMenu (mostrar mensajes y leer entradas) se repartieron entre ConsolaBase, LectorEntradas, las tres consolas y OperacionesComercio. La idea fue que la clase que lee y muestra datos no sea la misma que hace las operaciones, porque en el Proyecto 1 CoordinadorAplicacion hacía las dos cosas y era muy difícil de probar. Las responsabilidades nuevas del Proyecto 2 están al final de la tabla en amarillo.

Responsabilidad	Tipo	Asignada a
Validar si el inicio de sesión coincide con un usuario registrado y activo.	Decidir	Autenticador
Almacenar las credenciales de acceso, y el estado de cuenta de usuario.	Saber	Usuario
Almacenar datos de fecha de nacimiento y puntos.	Saber	Socio
Almacenar la disciplina a cargo y asignación del personal técnico.	Saber	Entrenador
Almacenar los datos de contacto y validación médica del especialista.	Saber	Fisioterapeuta
Almacenar los privilegios administrativos.	Saber	Administrador
Mantener las reglas de puntuación, alineación e indicadores de cada deporte.	Saber	Deporte
Almacenar los indicadores de rendimiento de un deportista o equipo en un partido.	Saber	Estadísticas
Calcula la actualización del puntaje del ranking, tras finalizar un encuentro deportivo.	Hacer	Ranking
Almacenar el registro de tarjetas amarillas y rojas y sanciones actuales de un jugador.	Saber	Sanción deportiva
Evaluar si un deportista presenta sanciones o acumulación de tarjetas que le impidan jugar.	Decidir	Verificación sanción

Organizar el avance de los competidores entre la fase de ligas y eliminación directa.	Decidir	Competencia
Contener la lista oficial de socios vinculados a un deporte de conjunto.	Saber	Equipo
Almacenar la fecha, lugar, competidores, marcador final, estado de un partido y estadísticas de partidos de práctica.	Saber	Partido
Calcular la tabla de posiciones ordenando los equipos según el criterio de cada deporte.	Hacer	Tabla de posiciones
Almacenar el registro y estado de inscripción de un equipo en una competencia.	Saber	Inscripción equipo
Almacenar la relación de un socio a un equipo incluyendo su número de camiseta, posición y nivel asignado al socio por la disciplina (principiante, intermedio o avanzado).	Saber	Inscripcion jugador
Evaluar la disponibilidad de cupos y requisitos de categoría antes de registrar una inscripción.	Decidir	Verificación inscripción
Verificar que los jugadores postulados a un partido pertenezcan al equipo, estén habilitados y sean de niveles iguales o adyacentes.	Decidir	Verificación alineación
Guardar el horario, objetivo, cancha y lista de asistencia en un entrenamiento.	Saber	Sesión de entrenamiento
Almacenar las condiciones físicas e indicadores registrados en un deportista.	Saber	Evaluación de rendimiento
Almacenar la fecha, horario, espacio y socio que reserva una instalación.	Saber	Reserva de instalación
Validar la disponibilidad del espacio y el máximo de instalaciones simultáneas por disciplina.	Decidir	Verificación reserva
Almacenar el nombre, cantidad y precio de un artículo en la tienda.	Saber	Producto
Almacenar los productos de la cafetería y especificar cuáles tienen alérgenos.	Saber	Alimento cafetería

Guardar la solicitudes de los productos sugeridos por los empleados	Saber	Sugerencia producto
Registrar y ejecutar el movimiento de artículos entre bodegas o ubicaciones del club.	Hacer	Inventario
Almacenar los productos comprados, fecha, cliente y costo total pagado en la transacción	Saber	Venta
Almacenar las condiciones de vigencia, porcentaje de los descuentos y asegurarse de que no haya acumulación de descuentos.	Saber	Gestor descuentos
Calcular los montos finales de una factura, aplicando las tasas del IVA o consumo.	Hacer	Impuestos
Almacenar los porcentajes de impuestos del IVA y consumo, tasa de puntos y descuentos.	Saber	Parámetro financiero
Almacenar el saldo acumulado de puntos y el historial de relaciones de un socio.	Saber	Cuenta puntos
Calcular y asignar puntos de fidelidad acumulados por transacciones de compra.	Decidir	Calculador puntos
Evaluar si al realizar una compra de una bebida caliente hay una reserva de canchas de pádel o mesa de tenis techada.	Decidir	Regla de bebida caliente
Almacenar la asignación de horario, rol y trabajador responsable.	Saber	Turnos
Almacenar el estado de un cambio de turno (pendiente, aprobada o rechazada), solicitado por un empleado.	Saber	Solicitud cambio
Verificar la cobertura mínima del personal en cada horario y evaluar la aprobación de cambio de turno.	Decidir	Registro turnos
Procesar los datos de las ventas, reservas y asistencias para crear informes.	Hacer	Generador reportes
Cargar y guardar los archivos de texto del sistema hacia y desde los archivos de texto locales.	Hacer	Almacenador de archivos

Iniciar sesión, mostrar el menú en ciclo, atrapar los errores de las operaciones y guardar los datos al salir.	Hacer	ConsolaBase
Leer cada dato que escribe el usuario y volver a pedirlo si está vacío, fuera de rango o con formato incorrecto.	Hacer	LectorEntradas
Mostrar las opciones de cada tipo de usuario y pedir los datos de cada operación.	Hacer	ConsolaAdministrador, ConsolaParticipante y ConsolaEmpleado
Saber qué operaciones puede pedirle la consola a la lógica.	Saber	FachadaTorneos
Recibir las peticiones de la consola, revisar el rol de quien las pide y repartirlas a las clases de la lógica.	Hacer	ClubFachada
Hacer las compras en tienda y cafetería, los cambios de turno y los reportes sin imprimir ni leer nada.	Hacer	OperacionesComercio
Avisar a la consola que una operación no se pudo hacer y por qué.	Saber	OperacionRechazadaExcepti on
Crear, buscar y listar los torneos, y decidir si un participante entra por cupo preferencial o regular.	Hacer	GestorTorneos
Saber el estado del torneo, sus cupos, sus inscritos y sus encuentros, y controlar las inscripciones, los resultados y el cierre.	Saber / Hacer	Torneo
Calcular cuánto paga un participante y repartir los premios según el tipo de torneo.	Hacer	TorneoRecreativo y TorneoCompetitivo
Armar las rondas de la liga todos contra todos.	Hacer	GeneradorCalendario
Llevar los puntos de cada participante y ordenarlos con el desempate.	Hacer	TablaTorneo
Saber si un bono ya se usó y aplicarlo a una compra de su dueño.	Saber / Hacer	BonoDescuento
Validar que el código de un estudiante esté matriculado.	Saber	RegistroMatriculas

Cargar y guardar usuarios, torneos, inscripciones, encuentros, premios y bonos en la carpeta datos/.	Hacer	PersistenciaClub

Justificación de asignaciones

1. Decisión de jugadores alineados en un partido.
   Se asigna a la clase verificación alineación, junto a la clase de verificación sanción. La clase equipo conoce su lista oficial de jugadores y el partido conoce quiénes asistieron, pero determinar si un jugador está habilitado exige evaluar las reglas de tarjetas amarillas y rojas acumuladas, o sanciones. Poner esta decisión en equipo o socio incumpliría la regla de responsabilidad única. Por esta razón, una clase de tipo controller consulta las sanciones en verificación sanción y toma la decisión antes del juego.

2. Decisión de regla de bebidas calientes en canchas de pádel o mesas techadas.
   Se asigna a la clase regla bebida caliente.
   Esta regla hace parte del módulo comercial por la venta de productos y el deportivo por la reserva de instalaciones. Asignarle la verificación a la clase de producto o a reservas instalación generaría una mezcla inadecuada de los módulos. La clase regla de bebidas calientes actúa como un filtro que consulta a reserva instalación la ubicación del socio y evalúa si la orden incluye productos que no se pueden consumir.

3. Decisión del cálculo en la tabla de posiciones y reglas de cada deporte.
   Tabla posiciones ejecuta un cálculo leyendo las reglas de la clase de deporte. Para cumplir con el RNF 1 (flexibilidad de agregar deportes sin modificar el código de torneos), la clase deporte almacena los criterios de otorgamiento de puntos. Tabla de posiciones toma los datos del marcador final de partidos y aplica la reglas de puntos de cada deporte.

4. Decisión de verificación de cobertura mínima del personal en los turnos de trabajo.
   Se le asigna a la clase de registro turnos, que hace parte del módulo de comercio y recursos. La clase Solicitud cambio turno únicamente almacena los datos de la petición y su estado. Corresponde a la clase de registro turnos evaluar si al aprobar el cambio de turno de los empleados se cumple el cupo mínimo de personal requerido por horario.

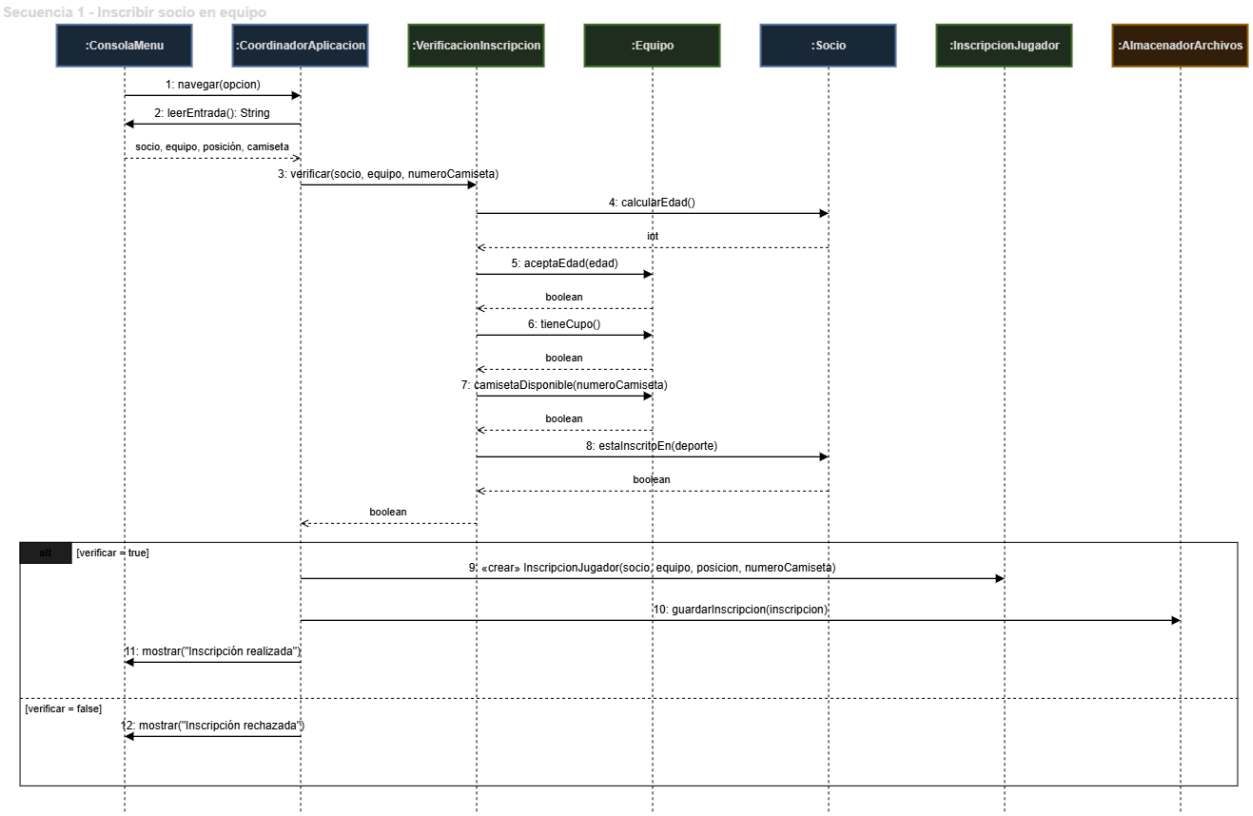
5. Colaboraciones: diagramas de secuencia y estilo de control

Se eligieron cuatro funcionalidades críticas, las que combinan varias clases de distintos componentes y aplican las reglas más delicadas del club. Los diagramas muestran que el diseño de clases alcanza para resolverlas, y todos sus mensajes son métodos del diagrama detallado de la sección 7 (los mensajes «crear» son llamadas a constructor).

Estos cuatro escenarios siguen siendo los mismos del Proyecto 1 y la lógica de sus diagramas no cambia. La diferencia es que donde aparece CoordinadorAplicacion, en el Proyecto 2 la petición la hace la consola correspondiente y la operación la ejecuta OperacionesComercio, que llama a las mismas clases en el mismo orden. Para el Proyecto 2 se agregaron dos secuencias de torneos (5.6 y 5.7), porque son las operaciones que combinan más reglas del componente nuevo.

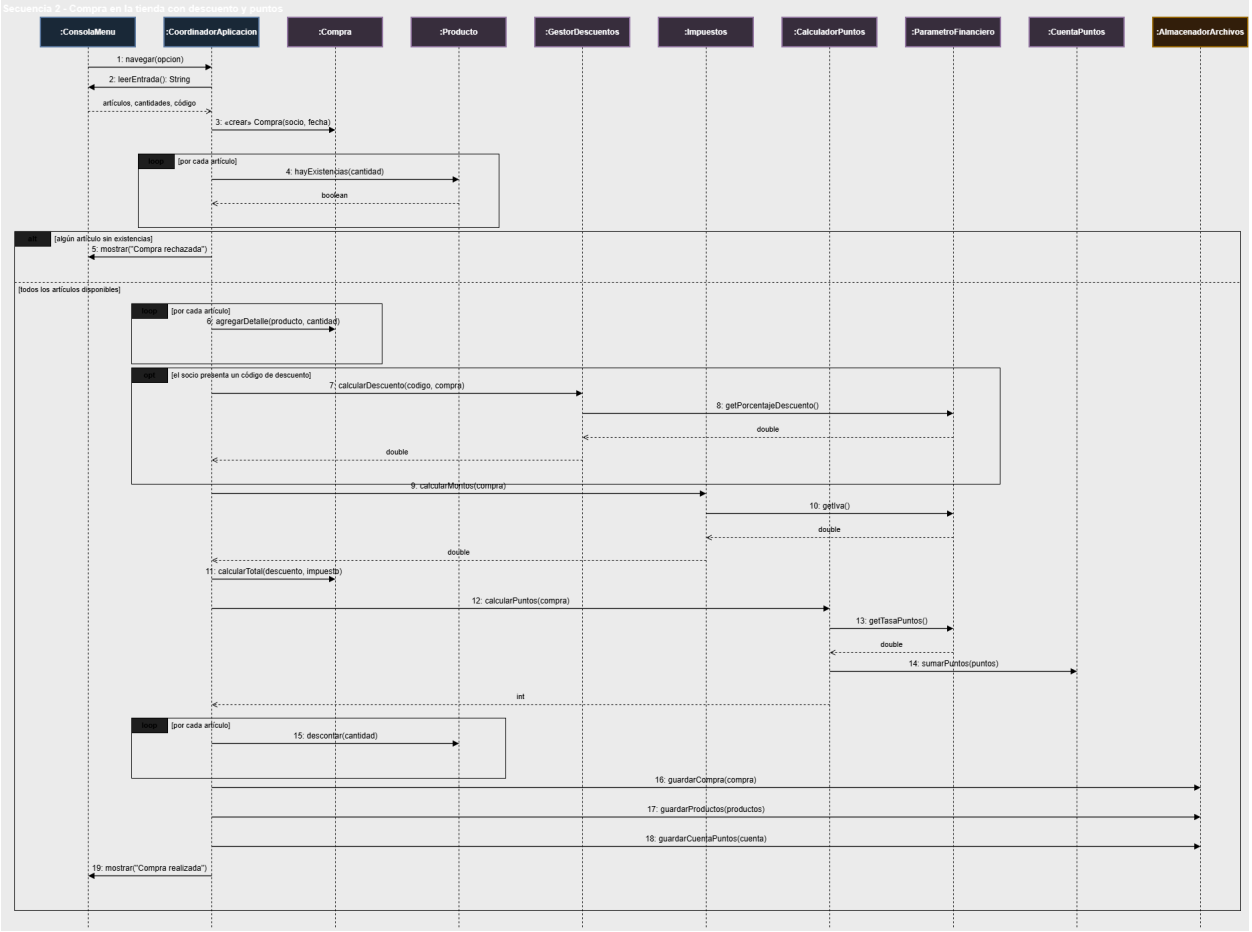
Escenario	Por qué es crítico	Quién decide
Inscribir a un socio en un equipo (RF1)	Aplica cuatro reglas que dependen de varias clases: edad, cupo, camiseta única y un solo equipo por deporte.	VerificacionInscripcion
Compra con descuento y puntos (RF9, RF12)	Reúne los tres cálculos financieros y es una transacción de varios artículos que no puede quedar a medias (RNF2 y RNF4).	Producto (existencias) y los calculadores
Registrar un partido con tarjetas (RF16, RF17, RF30, RF32)	Combina nómina, suspensiones, estadísticas y tabla de posiciones.	VerificacionAlineacion y VerificacionSancion
Venta con regla de bebida caliente (RF11)	Es la única regla que cruza el componente comercial y el deportivo.	ReglaBebidaCaliente
Inscribir a un jugador en un torneo (RF34, RF36, RF37)	Junta los cupos preferenciales por ranking, los turnos de los empleados, la categoría y el cobro según el tipo de torneo.	GestorTorneos y Torneo
Finalizar un torneo y entregar premios (RF39, RF40, RF41)	Ordena la tabla con desempate y reparte premios distintos según el tipo de torneo.	TorneoRecreativo y TorneoCompetitivo

5.1 Inscribir a un socio en un equipo



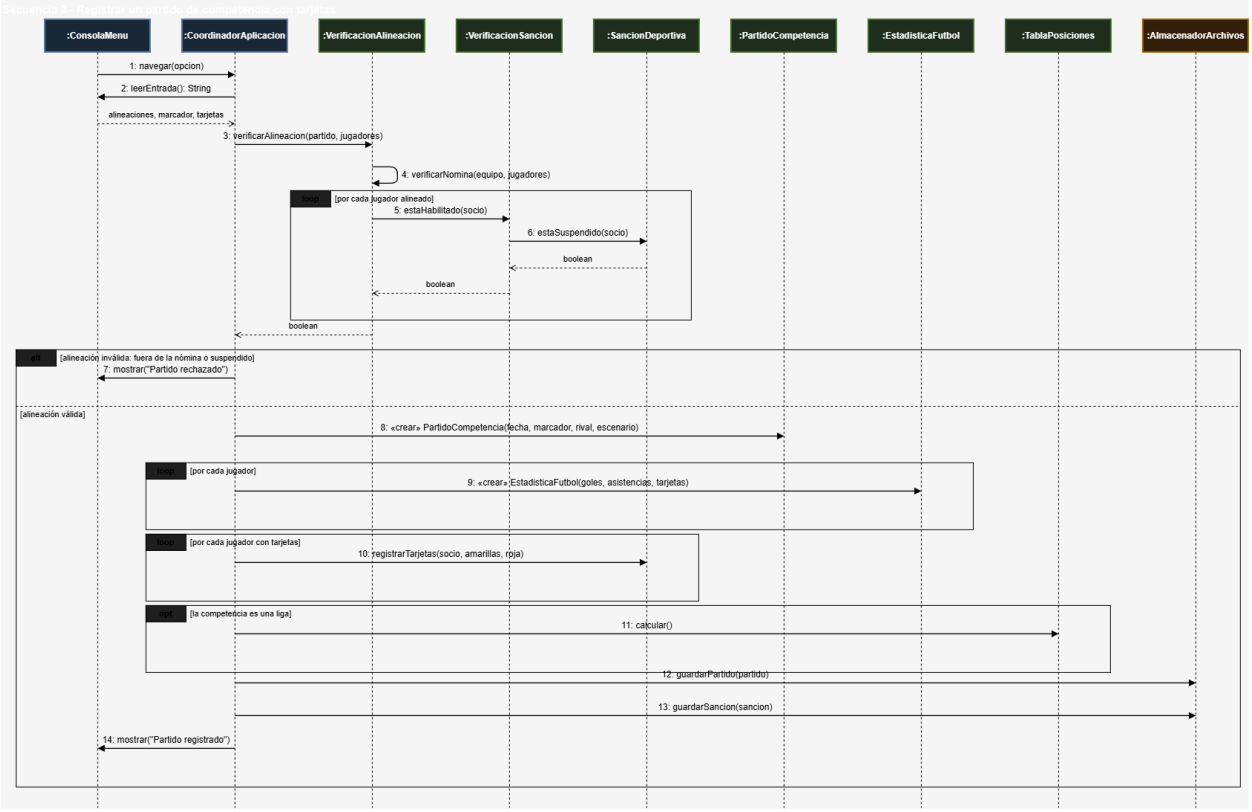
El coordinador llama a verificar(...) de VerificacionInscripcion, que le pregunta al Socio su edad y si ya está en un equipo de ese deporte, y al Equipo si acepta la edad, si tiene cupo y si el número de camiseta está libre. Si todo se cumple, el coordinador crea la InscripcionJugador y la guarda; si no, solo muestra el rechazo. La decisión está en una clase aparte para que ni Equipo ni Socio conozcan todas las reglas de inscripción.

5.2 Compra en la tienda con descuento y puntos



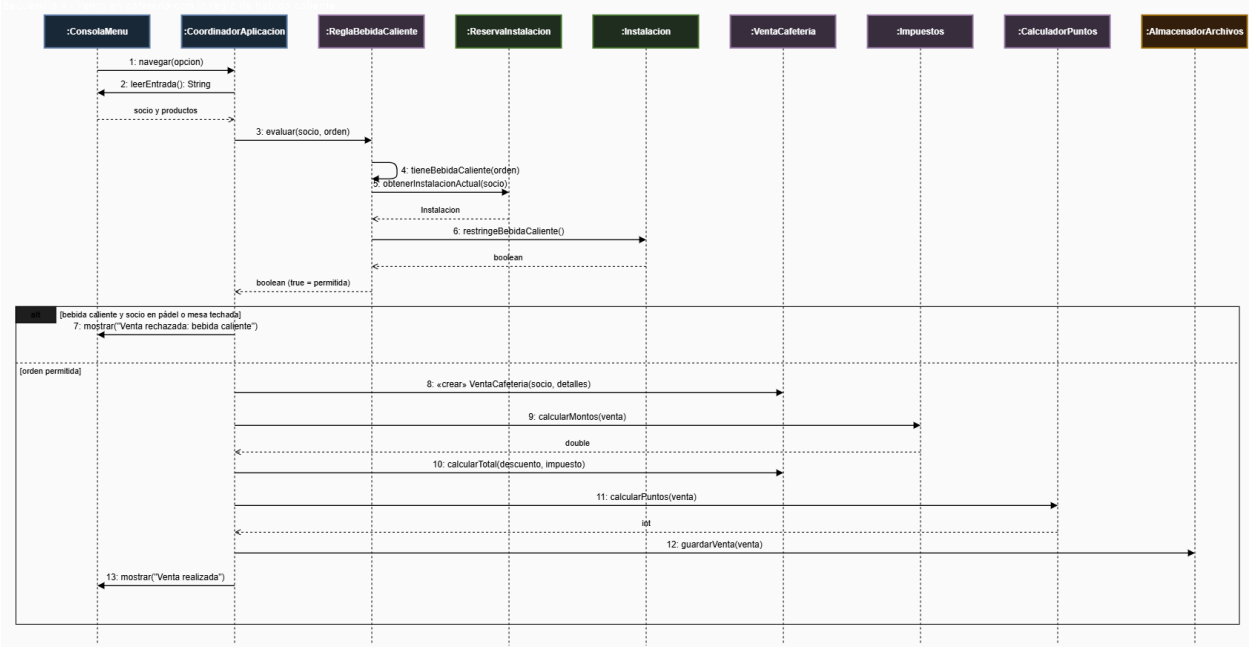
Primero se revisa que cada producto tenga existencias; si alguno no, la compra se rechaza completa y no se modifica nada. Si todos están disponibles, GestorDescuentos calcula el descuento (si hay código), Impuestos calcula el IVA y CalculadorPuntos calcula los puntos y los suma a la CuentaPuntos. Las tres consultan los porcentajes a ParametroFinanciero, así que cambiar el IVA o una tasa se hace en un solo lugar (RNF4). Solo al final se descuenta el inventario y se guarda. El orden (descuento, impuesto, puntos) es una decisión de diseño que puede cambiarse sin afectar las clases.

5.3 Registro de un partido de competencia con tarjetas



VerificacionAlineacion revisa la nómina y el mínimo de jugadores, y para cada jugador le pregunta a VerificacionSancion, que consulta a SancionDeportiva si está suspendido. Si la alineación es inválida el partido se rechaza. Si es válida se crean el partido y las estadísticas, se registran las tarjetas (2 amarillas o 1 roja suspenden al jugador), se calcula la tabla de posiciones cuando es una liga y se guarda todo.

5.4 Venta en cafetería con regla de bebida caliente



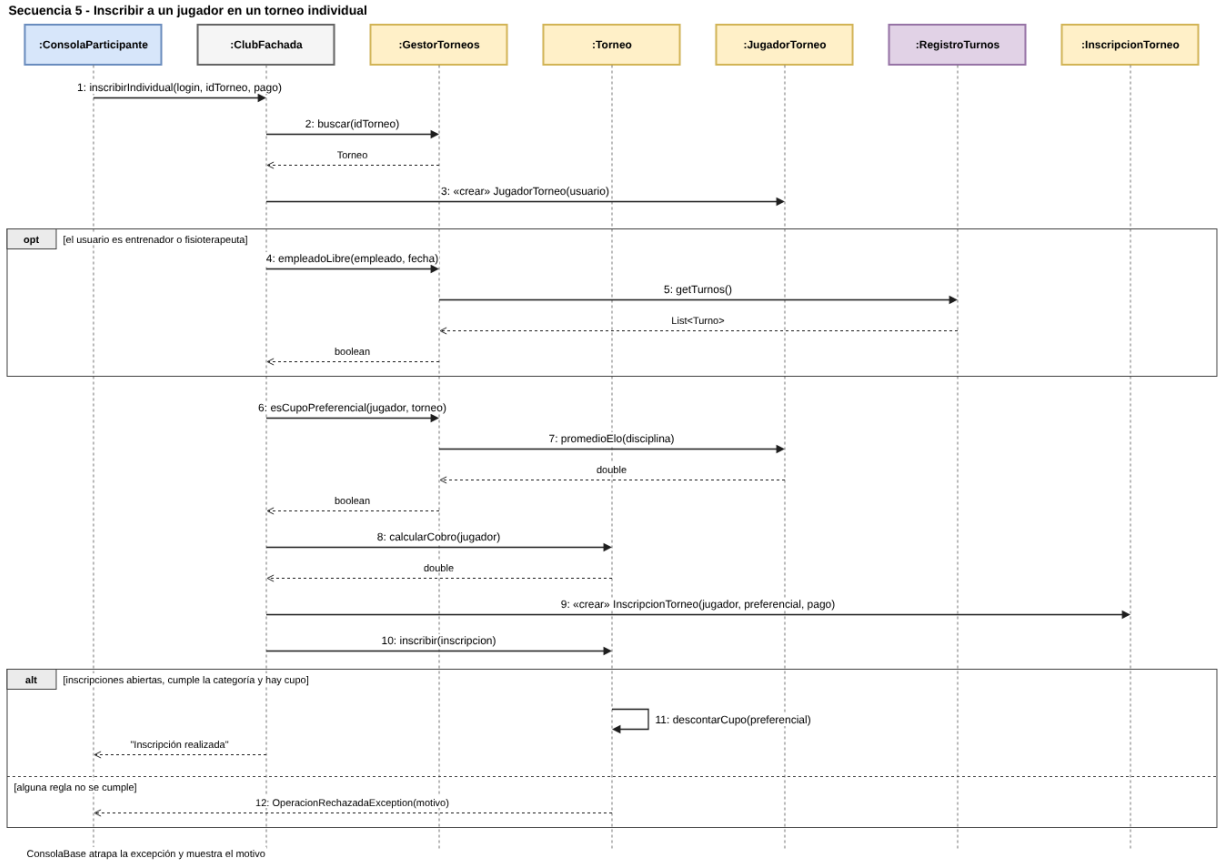
ReglaBebidaCaliente revisa si la orden tiene una bebida caliente y, si es así, le pide a ReservaInstalacion la instalación donde está el socio y le pregunta a esa Instalacion si restringe bebidas calientes (la cancha de pádel siempre, la mesa según si es techada). Si la restringe, la venta se rechaza; si no, se crea la venta, se calculan el impuesto y los puntos, y se guarda.

5.5 Estilo de control

El control es delegado. CoordinadorAplicacion recibe lo que pide el usuario y reparte el trabajo, pero no decide ni calcula: las decisiones las toman las clases Verificacion…, ReglaBebidaCaliente y RegistroTurnos, y los cálculos los hacen Impuestos, CalculadorPuntos y GestorDescuentos con los valores de ParametroFinanciero. Se descartó el control centralizado porque una sola clase gestora tendría que cambiar con cada regla y mezclaría reglas deportivas y comerciales, y el disperso porque las reglas quedarían repartidas entre Equipo, Socio y Partido. Lo que se sacrifica es que, para seguir una funcionalidad completa, hay que recorrer varias clases; por eso se hicieron estos diagramas.

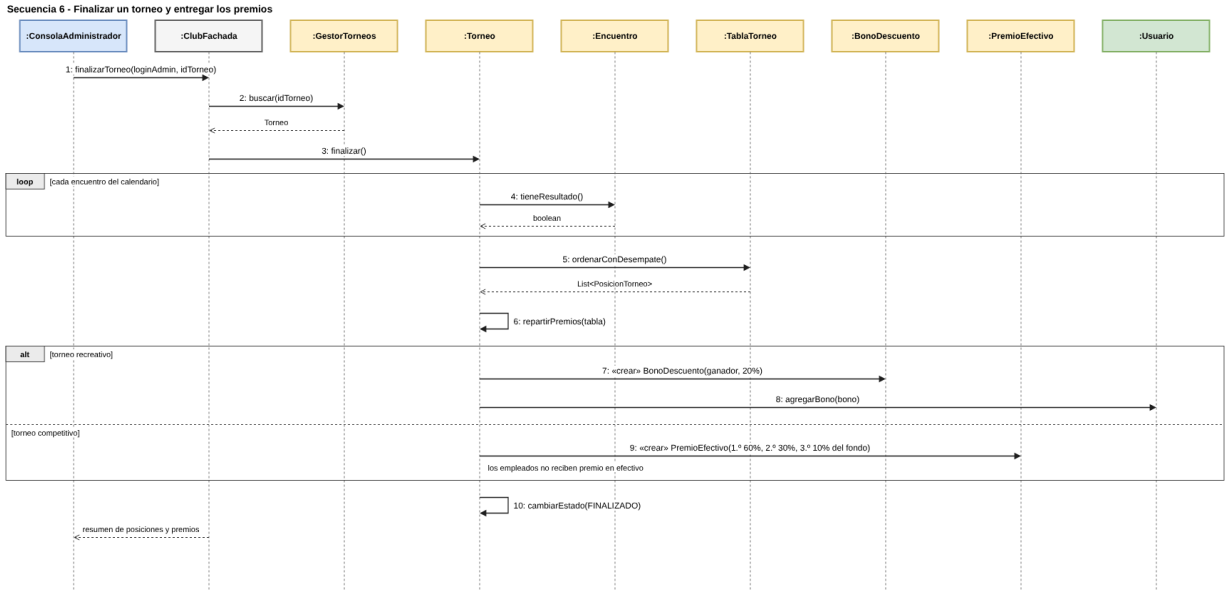
En el Proyecto 2 el estilo sigue siendo delegado, pero el papel de CoordinadorAplicacion se repartió: las consolas reciben lo que pide el usuario, ClubFachada reparte las peticiones de torneos y OperacionesComercio las de comercio. Las decisiones siguen estando en las clases de control de cada componente, como GestorTorneos para los cupos o Torneo para las inscripciones y el cierre.

5.6 Inscribir a un jugador en un torneo



ClubFachada busca el torneo y crea el participante (Jugador, Pareja o EquipoTorneo). Si el usuario es empleado, GestorTorneos revisa en RegistroTurnos que no tenga turno ese día. Después decide si el jugador entra por cupo preferencial revisando si su ELO está entre los mejores (top-K) de la disciplina, y el torneo calcula cuánto debe pagar (nada en un recreativo o si es empleado). Torneo hace las validaciones finales: que las inscripciones estén abiertas, que el jugador cumpla la categoría y que haya cupo. Si alguna falla lanza OperacionRechazadaException y la consola muestra el motivo.

5.7 Finalizar un torneo y entregar los premios



Torneo revisa que todos los encuentros tengan resultado y le pide a TablaTorneo las posiciones ordenadas con el desempate. Luego llama a repartirPremios, que es abstracto: TorneoRecreativo crea un BonoDescuento para cada ganador y se lo agrega al usuario, y TorneoCompetitivo reparte el fondo 60%, 30% y 10% entre los tres primeros con PremioEfectivo, sin darle efectivo a los empleados. Al final el torneo queda finalizado.

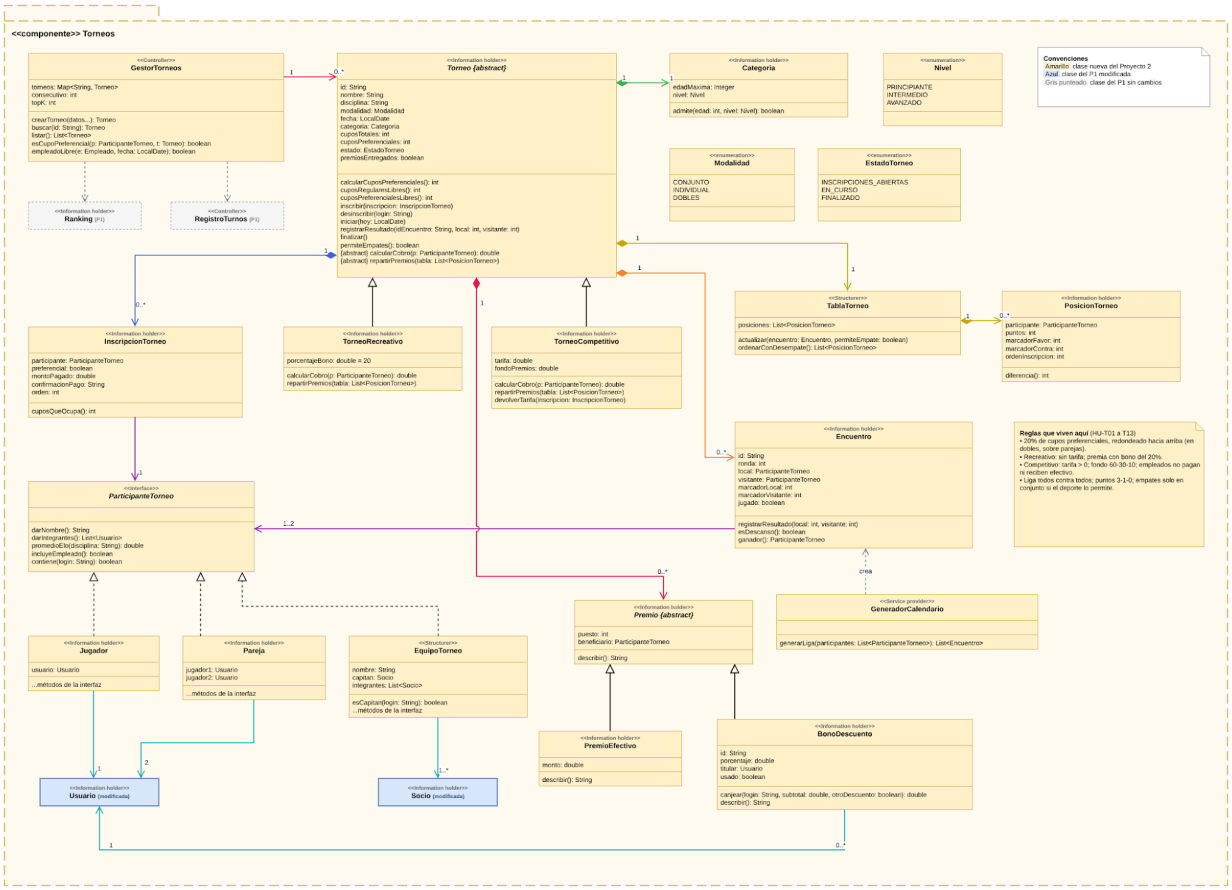
6. Diagrama de clases de alto nivel

Este diagrama muestra cómo quedan los componentes del Proyecto 2 y cuáles clases se agregaron o se modificaron. Para que se pudiera leer se dividió en cuatro páginas: una de alto nivel y una para cada parte que cambió (torneos, consola y fachada, y usuarios y persistencia). Los componentes que no cambiaron (Manejo deportivo y Comercio y recursos) mantienen los diagramas del Proyecto 1, que se dejan más abajo como referencia.

6.1 Especificaciones del diagrama:

Colores: amarillo para las clases nuevas del Proyecto 2, azul para las clases del Proyecto 1 que se modificaron y gris con borde punteado para las clases del Proyecto 1 que se dibujan solo como referencia.
Notación: se usa la misma que en el Proyecto 1. Herencia con flecha de triángulo vacío, implementación de interfaz con línea punteada y triángulo vacío, asociación con flecha simple y multiplicidad, composición con rombo relleno y dependencia con línea punteada.
El archivo editable es CDS_UML_Proyecto2.drawio, en la carpeta diagramas del repositorio.
Las primeras cuatro páginas son del Proyecto 2 y las siguientes son las del Proyecto 1 sin cambios.

6.2 Componente Torneos



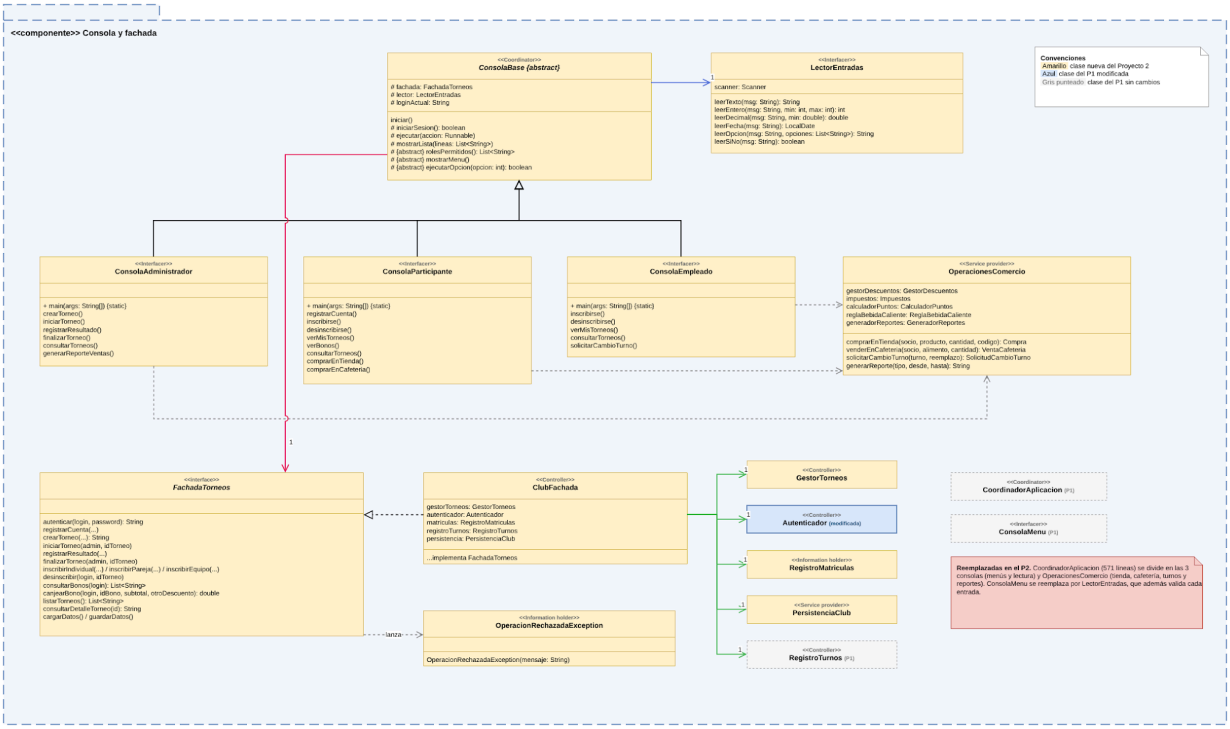
Es el componente nuevo y el más grande del Proyecto 2. GestorTorneos es el Controller: guarda todos los torneos, los crea y es el que decide si un participante entra por un cupo preferencial, porque para eso necesita consultar el Ranking del componente deportivo. También usa RegistroTurnos para saber si un empleado tiene turno el día del torneo.

Torneo es abstracta y de ella heredan TorneoRecreativo y TorneoCompetitivo. Lo que cambia entre los dos es cómo se cobra la inscripción y cómo se reparten los premios, por eso esos dos métodos son abstractos en Torneo. El recreativo entrega un BonoDescuento y el competitivo maneja el fondo y entrega PremioEfectivo. Las dos clases de premio heredan de Premio.

Lo que se inscribe en un torneo es un ParticipanteTorneo, que es una interfaz implementada por Jugador, Pareja y EquipoTorneo. Así el torneo, la tabla y el calendario no tienen que saber si están manejando una persona, una pareja o un equipo. InscripcionTorneo une al participante con el torneo y guarda si tomó cupo preferencial y cuánto pagó.

Cuando el torneo inicia, GeneradorCalendario arma los Encuentro de la liga todos contra todos, y con cada resultado se actualiza TablaTorneo, que tiene una PosicionTorneo por participante. La categoría del torneo (edad máxima o nivel) quedó en la clase Categoria.

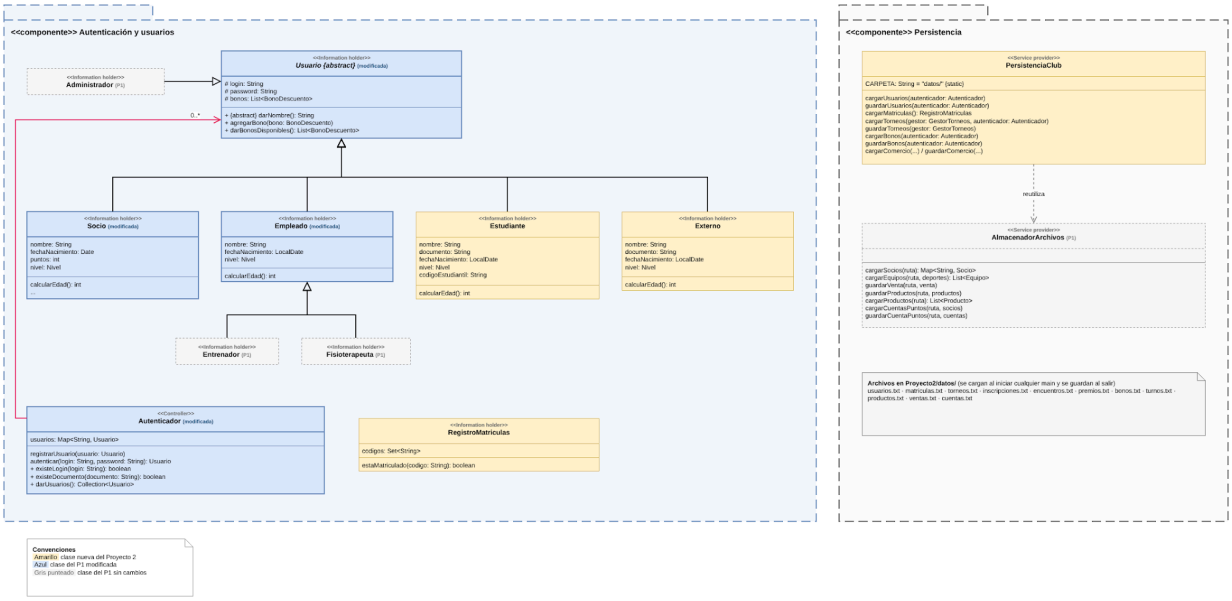
6.3 Componentes Consola y Fachada



La consola quedó con una clase abstracta ConsolaBase que tiene todo lo que se repite en los tres programas: iniciar sesión, el ciclo del menú, atrapar los errores y guardar al salir. ConsolaAdministrador, ConsolaParticipante y ConsolaEmpleado heredan de ella y cada una tiene su propio main. Ninguna de las consolas lee del teclado directamente, todas usan LectorEntradas.

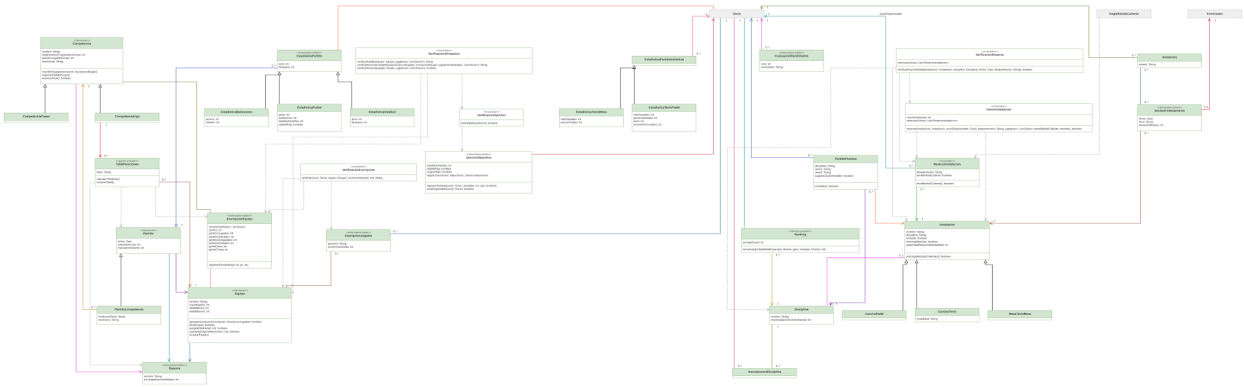
Las consolas solo conocen la interfaz FachadaTorneos y la clase OperacionesComercio. ClubFachada implementa la interfaz y es la que habla con GestorTorneos, Autenticador y PersistenciaClub. Cuando una regla del negocio no se cumple, la lógica lanza OperacionRechazadaException con un mensaje, y ConsolaBase lo atrapa y lo muestra sin que el programa se cierre.
OperacionesComercio salió de CoordinadorAplicacion. Tiene las mismas operaciones de tienda, cafetería, turnos y reportes del Proyecto 1, pero recibe los datos por parámetro y devuelve el resultado, en vez de pedirlos y mostrarlos ella misma.

6.4 Usuarios y Persistencia



A la jerarquía de usuarios del Proyecto 1 se le agregaron Estudiante y Externo, que heredan de Usuario. Usuario ahora guarda los bonos que gana cada persona, porque un bono lo puede ganar cualquier participante y no solo un socio. El estudiante tiene además su código estudiantil, que se valida contra RegistroMatriculas al crear la cuenta.
En persistencia se mantiene AlmacenadorArchivos con los archivos del Proyecto 1 y se agregó PersistenciaClub, que carga y guarda los usuarios, los torneos, las inscripciones, los encuentros, los premios y los bonos. Se dejó como una clase aparte para no seguir creciendo AlmacenadorArchivos, que ya tenía muchos métodos, y para que si cambia el formato de los archivos de torneos el cambio quede solo ahí (RNF3).

6.5 Componente Manejo deportivo (sin cambios)



Este componente no tuvo cambios en el Proyecto 2. Se dejan el diagrama y la explicación del Proyecto 1.

Es el más grande, por ende, se divide en cuatro bloques.

Equipos e inscripciones. Equipo está asociado con Deporte. InscripcionJugador une a un Socio con un Equipo y guarda la posición, el número de camiseta y el nivel; InscripcionEquipo une a un Equipo con una Competencia y guarda la nómina de jugadores habilitados. VerificacionInscripcion es la clase que decide si una inscripción es válida (edad, cupo, camiseta única, un solo equipo por deporte), por eso depende de Equipo, InscripcionJugador e InscripcionEquipo pero no guarda nada de ellas.

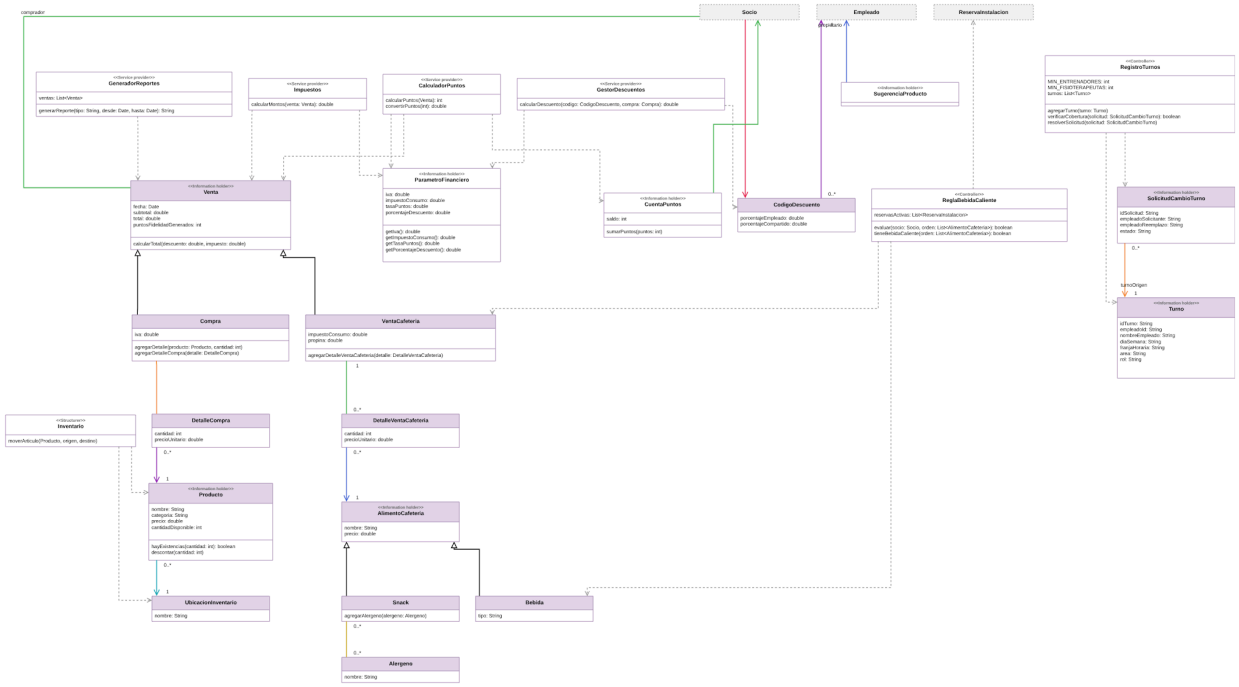
Competencias, partidos y tablas. Competencia es la clase padre de CompetenciaFases y CompetenciaLiga, y se asocia con Deporte. Partido se relaciona con los Equipos que juegan y con el Deporte, y PartidoCompetencia agrega la Competencia a la que pertenece. TablaPosiciones se asocia con una CompetenciaLiga y con los Equipos que aparecen en ella, y depende de Deporte y de Partido: lee las reglas de puntos del deporte y los marcadores para calcular las posiciones, que es lo que permite agregar un deporte sin tocar el código de los torneos (RNF1). VerificacionAlineacion depende de Equipo, Partido, InscripcionJugador y de VerificacionSancion, que a su vez consulta SancionDeportiva; con eso se rechaza la alineación de un jugador suspendido.

Estadísticas y ranking. EstadisticaPartido es la clase padre de EstadisticaFutbol, EstadisticaBaloncesto y EstadisticaVoleibol, y se asocia con el Partido y con el Socio. Para los deportes individuales, EstadisticaPartidoIndividual es el padre de EstadisticaTenisMesa y EstadisticaTenisPadel y se asocia con el Socio. Ranking se asocia con Disciplina y con Socio y se actualiza con el resultado de los partidos de práctica. Cada deporte nuevo se agrega como una subclase más en estas dos jerarquías.

Disciplinas, instalaciones y entrenamiento. Disciplina se relaciona con las Instalaciones donde se practica y con InscripcionesDisciplina, que a su vez apunta al Socio inscrito (RF2). Instalacion es la clase padre de MesaTenisMesa, CanchaPadel y CanchaTenis. ReservaInstalacion se asocia con el Socio y la Instalacion, y VerificacionReserva depende de ella, de Instalacion y de Disciplina para validar el máximo de instalaciones simultáneas.

SesionEntrenamiento se asocia con una Instalacion y un Entrenador, Asistencia relaciona a los Socios con cada sesión y EvaluacionRendimiento queda ligada al Socio evaluado. PartidoPractica se asocia con el Socio, la Instalacion y la Disciplina.

6.6 Componente Comercio y recursos (sin cambios)



Este componente tampoco cambió. La única diferencia es que las operaciones que antes hacía CoordinadorAplicacion ahora las hace OperacionesComercio (ver 6.3), pero las clases de esta página son las mismas.

Ventas. Venta es la clase padre de Compra (artículos deportivos de la tienda) y de VentaCafeteria. Cada una se compone de sus detalles: Compra tiene DetalleCompra, que apunta a un Producto, y VentaCafeteria tiene DetalleVentaCafeteria, que apunta a un AlimentoCafeteria. Bebida y Snack son subclases de AlimentoCafeteria, y Snack se relaciona con Alergeno (muchos a muchos). Venta también se asocia con el Socio que compra.

Inventario. Producto se asocia con la UbicacionInventario donde está. La clase Inventario depende de las dos para mover y reabastecer artículos. SugerenciaProducto queda ligada al Empleado que la propone.

Cálculos financieros. Aquí está la decisión más importante del componente: Impuestos, CalculadorPuntos y GestorDescuentos no tienen los valores escritos adentro, los toman de ParametroFinanciero (IVA, impuesto al consumo, tasa de puntos y porcentaje de descuento). Por eso las tres dependen de esa clase, y cambiar el IVA o un porcentaje implica editar un solo lugar (RNF4). Impuestos depende además de Venta, CalculadorPuntos de Venta y de CuentaPuntos (que se asocia con el Socio), y GestorDescuentos de CodigoDescuento, que está asociado con el Empleado dueño del código y con el Socio con quien lo comparte.

Turnos. Turno y SolicitudCambioTurno se asocian con Empleado; la solicitud apunta a dos turnos, el actual y el solicitado. RegistroTurnos depende de las dos y es la que verifica la cobertura mínima de personal antes de aprobar un cambio.

Regla de bebida caliente y reportes. ReglaBebidaCaliente depende de ReservaInstalacion (del componente deportivo), de Bebida y de VentaCafeteria, y es la única clase del comercio que consulta al componente deportivo, justo lo que dice la descripción del componente en el diagrama de componentes. GeneradorReportes depende de Venta, ReservaInstalacion y Asistencia, porque los reportes financieros y deportivos se arman leyendo esas clases.

6.7 Diferencias con el modelo de dominio de la entrega 1

- Se agregaron las 19 clases nuevas de la sección 4 que no estaban en el dominio (CoordinadorAplicacion, ConsolaMenu, Autenticador, VerificacionInscripcion, VerificacionAlineacion, SancionDeportiva, VerificacionSancion, VerificacionReserva, Inventario, SugerenciaProducto, GestorDescuentos, ParametroFinanciero, Impuestos, CuentaPuntos, CalculadorPuntos, ReglaBebidaCaliente, RegistroTurnos, GeneradorReportes y AlmacenadorArchivos).
- El nivel del jugador (principiante, intermedio o avanzado) quedó como atributo de InscripcionJugador, y InscripcionesDisciplina quedó solo como la unión entre Disciplina y Socio.
- La clase Categoria del dominio no se llevó como clase aparte: la edad mínima y máxima quedaron como atributos de Equipo y de Competencia, que es lo que piden las validaciones de RF1 y RF29.
- Las relaciones de uso de las clases de la sección 4 (verificaciones, calculadores, gestores) se dibujaron como dependencias, para que se vea que ellas consultan a las clases del dominio pero no las guardan.

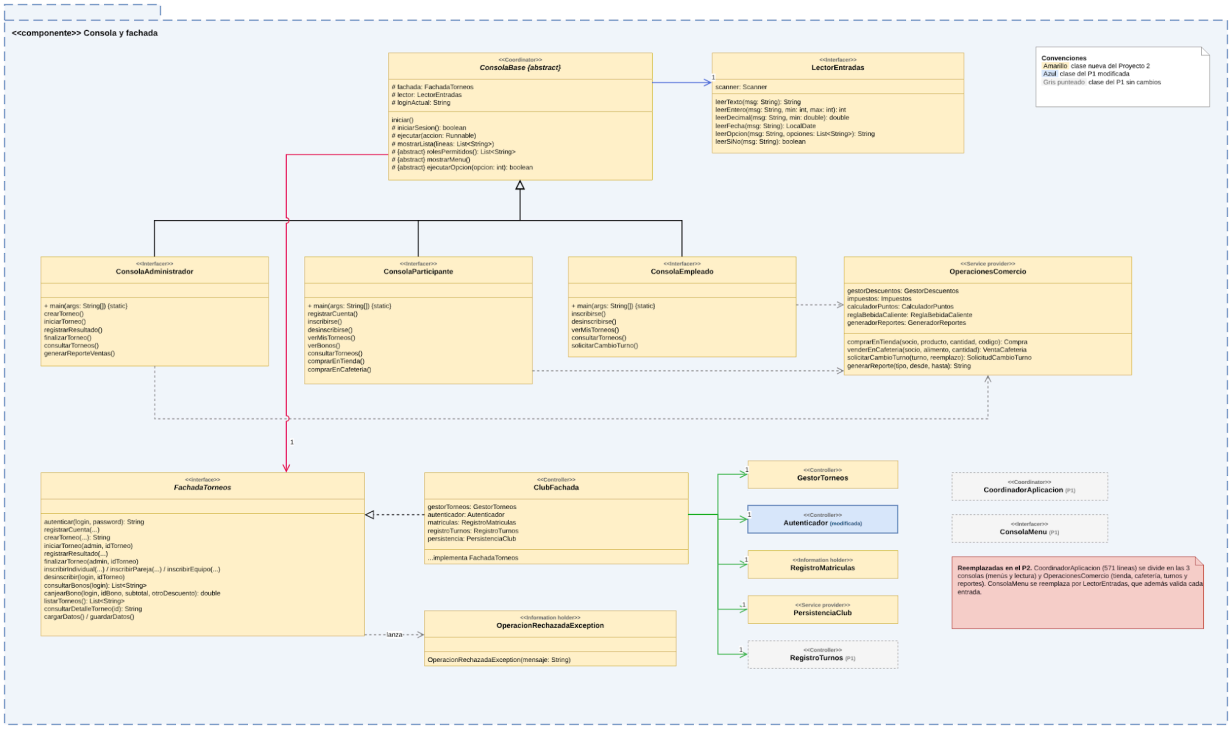
7. UML

El diagrama de diseño muestra lo indicado en la sección 6, pero con los atributos y los métodos (con nombre, parámetros y retorno) de cada clase. Para que se pueda leer se dividió en una página por componente: Autenticación y consola, Manejo deportivo y Comercio y recursos. Persistencia tiene una sola clase, AlmacenadorArchivos, y las cajas grises punteadas son clases de otro componente que se dibujan solo como referencia de hacia dónde va una línea; no son clases nuevas.

La notación es la de la sección 6. Los atributos de todas las clases están en el diagrama; en las tablas de esta sección se describen las clases más importantes del componente, es decir, las que toman decisiones, calculan o estructuran a otras, y se listan sus métodos. Las clases que solo guardan datos (por ejemplo los detalles de venta, el alérgeno o las instalaciones específicas) se explican por su nombre y sus atributos en el diagrama. Todos los mensajes de las secuencias de la sección 5 son métodos de estas clases.

Las tablas 7.2, 7.3 y 7.4 son las del Proyecto 1 porque esos componentes no cambiaron. La tabla de la consola se reemplazó por la del Proyecto 2 (7.1) y se agregaron las tablas de torneos (7.6) y de usuarios y persistencia nuevos (7.7).

7.1 Consola y fachada (Proyecto 2)

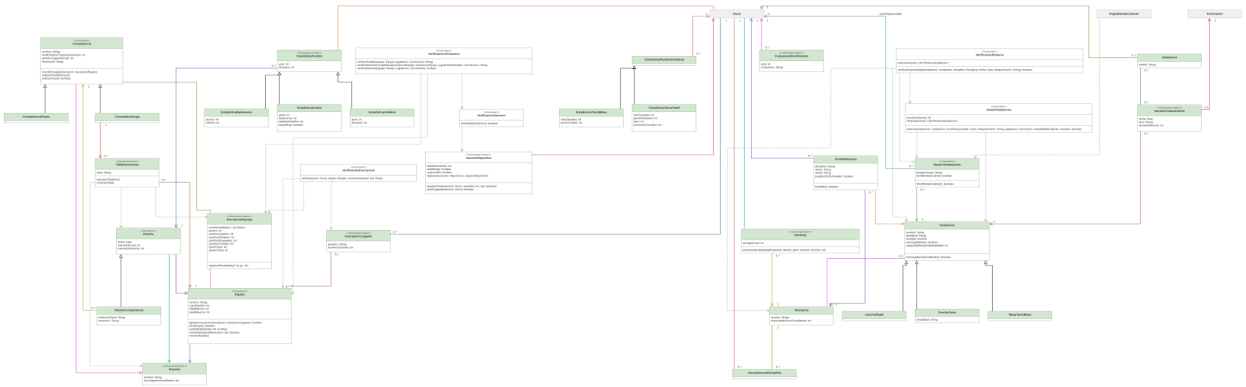


Clase	Estereotipo	Descripción	Métodos clave
Autenticador	Controller	Valida el usuario y la contraseña al ingresar y registra el rol correspondiente.	autenticar(login: String, password: String): Usuario
Usuario (abstracta)	Information holder	Almacena los datos base de identificación (login y contraseña) de todas las personas registradas.	-
Socio	Information holder	Persona inscrita en el club; guarda su nombre, fecha de nacimiento y puntos, y sabe calcular su edad.	calcularEdad(): int estaInscritoEn(deporte: Deporte): boolean
ConsolaBase	Coordinator	Clase abstracta con el inicio de sesión, el ciclo del menú, el manejo de errores y el guardado al salir.	iniciar() iniciarSesion(): boolean ejecutar(accion: Runnable) mostrarMenu() {abstract} ejecutarOpcion(opcion: int): boolean {abstract}
LectorEntradas	Interfacer	Lee lo que escribe el usuario y lo vuelve a pedir hasta que sea válido.	leerTexto(msg): String leerEntero(msg, min, max): int leerDecimal(msg, min): double leerFecha(msg): LocalDate leerSiNo(msg): boolean
ConsolaAdministrador	Interfacer	Menú del administrador para manejar torneos y ver el reporte de ventas.	main(args) crearTorneo() iniciarTorneo() registrarResultado() finalizarTorneo()
ConsolaParticipante	Interfacer	Menú de socios, estudiantes y externos. Permite crear una cuenta antes de iniciar sesión.	main(args) registrarCuenta() inscribirse() verBonos() comprarEnTienda()

ConsolaEmpleado	Interfacer	Menú de entrenadores y fisioterapeutas para torneos y cambios de turno.	main(args) inscribirse() solicitarCambioTurno()
FachadaTorneos	Interfacer	Interfaz con las operaciones que la consola puede pedirle a la lógica.	autenticar(login, password): String crearTorneo(...): String inscribirIndividual(...) finalizarTorneo(admin, id) canjearBono(...): double
ClubFachada	Controller	Implementa la fachada y reparte las peticiones a la lógica.	(los de la interfaz)
OperacionesComercio	Service provider	Compras, ventas, cambios de turno y reportes del Proyecto 1, sin entrada ni salida por consola.	comprarEnTienda(...) venderEnCafeteria(...) solicitarCambioTurno(turno, reemplazo) generarReporte(tipo, desde, hasta): String

Ninguna de estas clases decide reglas del club. Las consolas piden datos y muestran resultados, la fachada reparte y OperacionesComercio solo llama a las clases de Comercio y recursos que ya existían (GestorDescuentos, ReglaBebidaCaliente, RegistroTurnos y GeneradorReportes).

7.2 Manejo deportivo



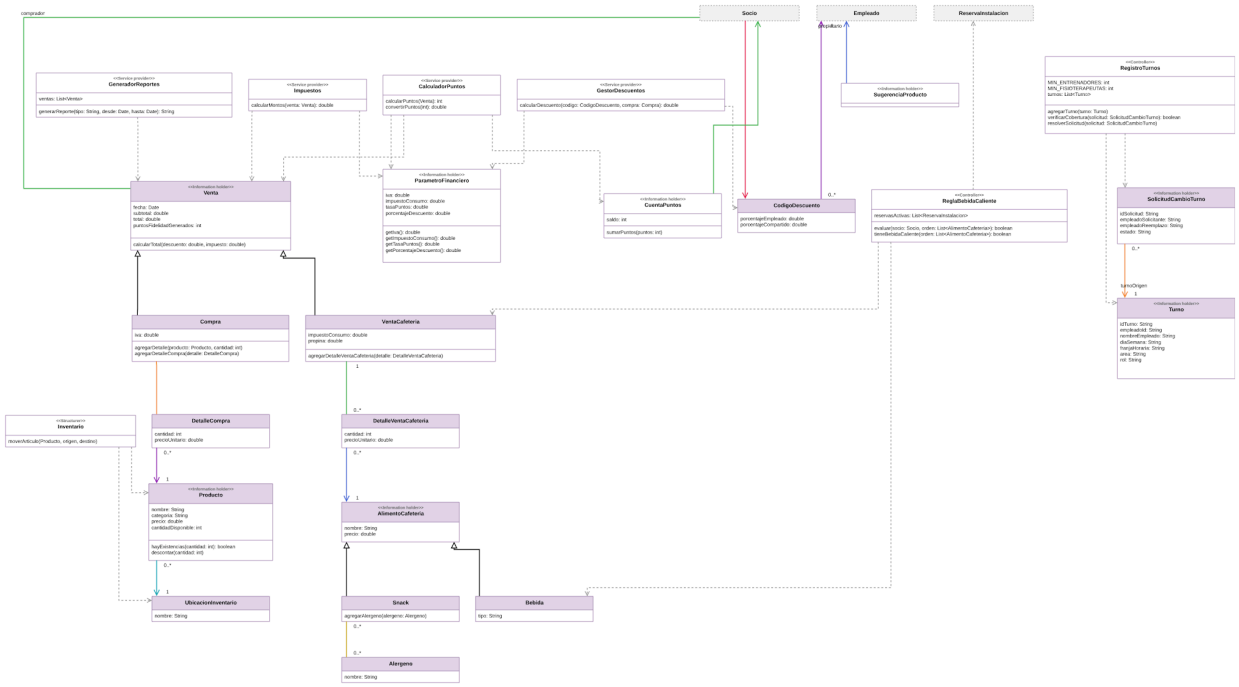
Clase	Estereotipo	Descripción	Métodos clave

Deporte	Information holder	Representa una disciplina y define sus reglas de puntuación y el mínimo de jugadores habilitados.	-
Equipo	Structurer	Agrupa a los socios de un deporte grupal que están habilitados para competir; conoce su cupo y su rango de edad.	tieneCupo(): boolean aceptaEdad(edad: int): boolean camisetaDisponible(numero: int): boolean
InscripcionJugador	Information holder	Registra la inscripción de un socio en un equipo, con su posición, número de camiseta y nivel.	-
InscripcionEquipo	Information holder	Registra la inscripción de un equipo en una competencia y su nómina de jugadores habilitados.	-
VerificacionInscripcion	Controller	Verifica la edad, el cupo, la camiseta única y el equipo único por deporte antes de inscribir a un socio.	verificar(socio: Socio, equipo: Equipo, numeroCamiseta: int): boolean
Competencia	Structurer	Organiza la estructura, las fases y los equipos de un torneo o liga.	avanzarFase()
TablaPosiciones	Service provider	Calcula y ordena los puntos y el rendimiento de los equipos de una liga según las reglas de puntos del deporte.	calcular()
VerificacionAlineacion	Controller	Verifica que los jugadores alineados estén en la nómina, que se alcance el mínimo de jugadores y que ninguno esté suspendido.	verificarAlineacion(partido: Partido, jugadores: List<Socio>): boolean verificarNomina(equipo: Equipo, jugadores: List<Socio>): boolean
VerificacionSancion	Controller	Verifica si un jugador está habilitado o suspendido por tarjetas acumuladas.	estaHabilitado(Socio): boolean

SancionDeportiva	Information holder	Almacena y cuenta las tarjetas y las suspensiones de un jugador.	registrarTarjetas(socio: Socio, amarillas: int, roja: boolean) estaSuspendido(socio: Socio): boolean
EstadisticaPartido	Information holder	Almacena los indicadores de un jugador en un partido de equipo; es la clase padre de las estadísticas de cada deporte de equipo.	-
Ranking	Service provider	Calcula y actualiza el puntaje de cada jugador por disciplina según los resultados.	actualizarPuntaje(socio: Socio, resultado: int)
Instalacion	-	Espacio deportivo con identificador y capacidad máxima; es la clase padre de las mesas y canchas.	restringeBebidaCaliente(): boolean
ReservaInstalacion	Controller	Administra la asignación de instalaciones a los socios por fecha y bloque horario, y sabe en qué instalación está un socio.	obtenerInstalacionActual(socio: Socio): Instalacion
VerificacionReserva	Controller	Verifica la disponibilidad de la instalación y el máximo de instalaciones simultáneas de la disciplina.	verificarDisponibilidad(instalacion: Instalacion, disciplina: Disciplina, fecha: Date, bloqueHorario: String): boolean

Las reglas complejas están en las clases Verificacion, que consultan a Equipo, Socio, Partido y las demás sin modificarlas. Un socio se relaciona con un equipo a través de InscripcionJugador (posición, camiseta y nivel), lo que conserva el historial. Las estadísticas, las instalaciones y las competencias se organizan con herencia (por ejemplo EstadisticaFutbol, CanchaPadel o CompetenciaLiga), de modo que un deporte nuevo se agrega con clases nuevas (RNF1).

7.3 Comercio y recursos



Clase	Estereotipo	Descripción	Métodos clave
Producto	Information holder	Guarda el nombre, la categoría, el precio y la cantidad disponible de un artículo, y descuenta lo vendido.	hayExistencias(cantidad: int): boolean descontar(cantidad: int)
Venta	Information holder	Registra los datos de una transacción (fecha, subtotal, total y puntos generados) y calcula su total.	calcularTotal(descuento: double, impuesto: double)
Inventario	Structurer	Mueve artículos entre ubicaciones y mantiene las existencias.	moverArticulo(Producto, origen, destino)
ParametroFinanciero	Information holder	Almacena los valores del IVA, el impuesto al consumo, la tasa de puntos y el porcentaje de descuento.	getIva(): double getImpuestoConsumo(): double getTasaPuntos(): double getPorcentajeDescuento(): double

Impuestos	Service provider	Calcula los montos de IVA o de impuesto al consumo de una venta.	calcularMontos(venta: Venta): double
Calculador Puntos	Service provider	Calcula los puntos que genera una venta y convierte puntos a dinero.	calcularPuntos(Venta): int convertirPuntos(int): double
GestorDescuentos	Service provider	Calcula el descuento que corresponde según el código y el usuario que lo usa.	calcularDescuento(codigo: CodigoDescuento, compra: Compra): double
CuentaPuntos	Information holder	Almacena el saldo de puntos de fidelización de un socio.	sumarPuntos(puntos: int)
ReglaBebidaCaliente	Controller	Verifica que no se venda una bebida caliente a un socio dentro de una cancha de pádel o una mesa de tenis de mesa techada.	evaluar(socio: Socio, orden: List<AlimentoCafeteria>): boolean tieneBebidaCaliente(orden: List<AlimentoCafeteria>): boolean
RegistroTurnos	Controller	Verifica la cobertura mínima de personal y aprueba o rechaza los cambios de turno.	verificarCobertura(solicitud: SolicitudCambioTurno): boolean resolverSolicitud(solicitud: SolicitudCambioTurno)
Generador Reportes	Service provider	Procesa los datos de ventas, reservas y asistencias para crear los reportes administrativos.	generarReporte(tipo: String, desde: Date, hasta: Date): String

Venta es la clase padre de Compra (tienda) y VentaCafeteria, y cada una se compone de sus detalles. Impuestos, CalculadorPuntos y GestorDescuentos consultan sus valores a ParametroFinanciero, de modo que cambiar el IVA o un porcentaje se hace en un solo lugar (RNF4). ReglaBebidaCaliente es la única clase comercial que consulta al componente deportivo.

7.4 Persistencia

Clase	Estereotipo	Descripción	Métodos clave

AlmacenadorArchivos	Interfacer	Lee y escribe la información del sistema en los archivos de texto.	cargarDatos() guardarInscripcion(inscripcion: InscripcionJugador) guardarCompra(compra: Compra) guardarVenta(venta: VentaCafeteria) guardarProductos(productos: List<Producto>) guardarCuentaPuntos(cuenta: CuentaPuntos) guardarPartido(partido: PartidoCompetencia) guardarSancion(sancion: SancionDeportiva)

Es la única clase que conoce el formato de los archivos de texto (RNF3): carga los datos al iniciar y tiene un método de guardado por cada tipo de objeto que cambia en las funcionalidades críticas.

7.5 Relaciones y requerimientos no funcionales

Hay 97 relaciones: 19 de herencia, 31 asociaciones dirigidas, 14 asociaciones simples y 33 dependencias. Las dependencias son sobre todo de las verificaciones y los calculadores, que consultan a otras clases sin guardarlas. Las multiplicidades son las del modelo de dominio; en las pocas relaciones donde ese modelo no las mostraba con claridad no se colocó ninguna. Los requerimientos no funcionales se cubren así:

Requerimiento	**Diseño**
RNF1	Herencia en estadísticas, instalaciones y competencias; un deporte nuevo se agrega con clases nuevas.
RNF2	Las verificaciones deciden antes de crear o modificar algo, y la compra es una sola transacción.
RNF3	Solo AlmacenadorArchivos conoce el formato de los archivos.
RNF4	ParametroFinanciero concentra IVA, impuesto al consumo, tasa de puntos y descuento.
RNF5	Las tres consolas son las únicas clases que muestran mensajes, y solo conocen FachadaTorneos y OperacionesComercio. La lógica no imprime nada.

RNF6	LectorEntradas es la única clase que lee del teclado y repite la pregunta hasta que el dato sea válido. ConsolaBase atrapa OperacionRechazadaException y muestra el mensaje.
RNF7	ConsolaBase carga los datos al iniciar y los guarda al salir. PersistenciaClub y AlmacenadorArchivos usan rutas fijas de la carpeta datos/.
RNF8	La lógica se usa a través de FachadaTorneos y no lee ni imprime nada, por eso se puede probar con JUnit sin la consola.

7.6 Torneos (Proyecto 2)

Clase	**Estereotipo**	**Descripción**	**Métodos clave**
GestorTorneos	Controller	Guarda los torneos y decide el tipo de cupo de cada participante.	crearTorneo(datos...): Torneo buscar(id): Torneo esCupoPreferencial(p, t): boolean empleadoLibre(e, fecha): boolean
Torneo	Information holder	Clase abstracta con los datos, los cupos, el estado, las inscripciones y los encuentros.	calcularCuposPreferenciales(): int inscribir(inscripcion) desinscribir(login) iniciar(hoy) registrarResultado(id, local, visitante) finalizar()
TorneoRecreativo	Information holder	Torneo sin tarifa que premia con bono.	calcularCobro(p): double repartirPremios(tabla)
TorneoCompetitivo	Information holder	Torneo con tarifa y fondo que reparte en efectivo.	calcularCobro(p): double repartirPremios(tabla) devolverTarifa(inscripcion)
ParticipanteTorneo	Interfacer	Interfaz común de jugador, pareja y equipo.	darNombre(): String darIntegrantes(): List promedioElo(disciplina): double contiene(login): boolean
GeneradorCalendario	Service provider	Arma las rondas todos contra todos.	generarLiga(participantes): List

TablaTorneo	Structurer	Lleva las posiciones y aplica el desempate.	actualizar(encuentro, permiteEmpate) ordenarConDesempate(): List
Encuentro	Information holder	Partido del torneo con su ronda y marcador.	registrarResultado(local, visitante) esDescanso(): boolean ganador(): ParticipanteTorneo
BonoDescuento	Information holder	Bono del 20% de un solo uso.	canjear(login, subtotal, otroDescuento): double

7.7 Usuarios y persistencia (Proyecto 2)

Clase	**Estereotipo**	**Descripción**	**Métodos clave**
Usuario	Information holder	Clase abstracta del Proyecto 1, ahora guarda los bonos del usuario.	darNombre(): String {abstract} agregarBono(bono) darBonosDisponibles(): List
Estudiante	Information holder	Estudiante de la universidad con código estudiantil.	(heredados)
Externo	Information holder	Persona externa a la universidad.	(heredados)
RegistroMatriculas	Information holder	Códigos de los estudiantes matriculados.	estaMatriculado(codigo): boolean
Autenticador	Controller	Del Proyecto 1, ahora también revisa que el login y el documento no estén repetidos.	registrarUsuario(usuario) autenticar(login, password): Usuario existeLogin(login): boolean existeDocumento(documento): boolean
PersistenciaClub	Service provider	Carga y guarda los datos nuevos del Proyecto 2.	cargarUsuarios(autenticador) guardarUsuarios(autenticador) cargarTorneos(gestor, autenticador) guardarTorneos(gestor)

			cargarBonos(...) / guardarBonos(...)

Estas tablas coinciden con el código entregado. Las pocas diferencias con el diagrama (métodos de apoyo que se agregaron al implementar) se explican en la sección 11.3.

8. Decisiones de diseño

Esta sección reúne las decisiones que más cambiaron la estructura del sistema. De cada una se presenta el problema, las alternativas que se consideraron, la que se eligió, lo que se gana con ella y lo que se sacrifica, y el requerimiento con el que se relaciona. Las cuatro decisiones que se justificaron en la sección 4 sobre responsabilidades difíciles (alineación, bebida caliente, tabla de posiciones y cobertura de turnos) se retoman aquí dentro de las decisiones D1, D2 y D5, ya con las alternativas y lo que se sacrifica.

D1. ¿Quién decide las reglas complejas del club?

Problema	Reglas como la inscripción en un equipo (edad, cupo, camiseta única, un solo equipo por deporte), la alineación de un partido (nómina, mínimo de jugadores, suspensiones) o la cobertura mínima de personal en los turnos involucran datos de varias clases. Hay que decidir en qué clase se escribe cada una.
Alternativas	A) Que cada clase de datos valide lo suyo: Equipo valida la inscripción, Partido valida la alineación, SolicitudCambioTurno valida la cobertura. B) Una clase de control por cada regla compleja (VerificacionInscripcion, VerificacionAlineacion con VerificacionSancion, RegistroTurnos) que consulta a las clases de datos y devuelve la decisión. C) Una sola clase que valide todas las reglas del club.
Elegida	B
Ganamos	Equipo, Socio, Partido y SolicitudCambioTurno solo guardan y entregan datos, y una regla que cambia se modifica en una sola clase. Las verificaciones se hacen antes de crear nada, y por eso un rechazo no deja cambios a medias (RNF2).
Sacrificamos	Más clases en el diseño, y cada regla tiene que consultar los datos a otras clases en lugar de leerlos de sí misma. La opción A habría tenido menos clases, pero cada clase de datos habría quedado con reglas de otros temas (por ejemplo, Equipo conocería las tarjetas de los jugadores), y la C habría sido una clase que crece con cada regla nueva.
Relacionada con	RF1, RF16, RF25, RF30 y RF32; RNF2. Decisiones 1 y 4 de la sección 4.

D2. ¿Cómo agregar un deporte nuevo sin modificar torneos, rankings ni estadísticas?

Problema	El club tiene varios deportes con estadísticas, puntos e instalaciones distintas, y se pueden agregar más. Los torneos, el ranking y el registro de estadísticas no deberían cambiar cada vez.
Alternativas	A) Una sola clase de estadística con todos los atributos posibles y un atributo de tipo de deporte, con condicionales en los torneos y en la tabla de posiciones. B) Jerarquías de herencia: EstadisticaPartido (fútbol, baloncesto, voleibol), EstadisticaPartidoIndividual (tenis de mesa, tenis y pádel), Instalacion (mesa, cancha de pádel y de tenis) y Competencia (fases, liga), con las reglas de puntos guardadas en Deporte.
Elegida	B
Ganamos	Un deporte nuevo se agrega con clases nuevas y sin modificar las existentes. TablaPosiciones aplica las reglas de puntos que lee de Deporte y no tiene condicionales por deporte, y cada estadística tiene solo los atributos que le corresponden.
Sacrificamos	Más clases, y para entender el cálculo de una tabla hay que mirar Deporte, Partido y TablaPosiciones. Además, la jerarquía fija la forma de las estadísticas: si un deporte necesitara una estructura muy distinta, habría que revisar la clase padre.
Relacionada con	RNF1; RF5, RF8, RF31. Decisión 3 de la sección 4.

D3. ¿Dónde viven el IVA, el impuesto al consumo, los puntos y los descuentos?

Problema	Las compras de la tienda pagan IVA, las ventas de la cafetería pagan impuesto al consumo, cada venta genera puntos y los códigos de descuento dan un porcentaje. Estos valores pueden cambiar, y el flujo de las transacciones no debe cambiar con ellos.
Alternativas	A) Que cada clase tenga su valor: Compra con el IVA, VentaCafeteria con el impuesto, CalculadorPuntos con la tasa. B) Una clase ParametroFinanciero que guarda los cuatro valores y que consultan Impuestos, CalculadorPuntos y GestorDescuentos.
Elegida	B
Ganamos	Cambiar el IVA o un porcentaje implica editar un solo lugar y el flujo de la compra no cambia. Se puede verificar como pide el requerimiento: se cambia el IVA y se comprueba que solo se tocó esa clase.

Sacrificamos	Una clase y una consulta más en cada cálculo, y las tres clases de servicio dependen de ParametroFinanciero. Además, Compra y VentaCafeteria conservan el monto de iva e impuestoConsumo de cada venta como dato histórico, que es distinto del porcentaje vigente.
Relacionada con	RNF4; RF9, RF10, RF12.

D4. ¿Quién lee y escribe los archivos?

Problema	El sistema guarda sus datos en archivos de texto. Si el formato cambia, no debería afectar a las reglas del negocio.
Alternativas	A) Que cada clase lea y escriba su propia información. B) Una clase de persistencia (AlmacenadorArchivos), en su propio componente, que es la única que conoce los archivos y a la que el coordinador le pasa los objetos que hay que guardar.
Elegida	B
Ganamos	Ninguna clase del dominio sabe nada de archivos, y un cambio de formato (o de archivos a una base de datos) solo toca esa clase. También permite probar las reglas sin leer archivos.
Sacrificamos	El coordinador tiene que pasarle a AlmacenadorArchivos los objetos que se guardan, y la clase de persistencia conoce muchas clases del modelo (un método de guardado por tipo de objeto), lo que la hace crecer con el sistema.
Relacionada con	RNF3; restricción de almacenamiento en archivos de texto.

D5. ¿Dónde se decide la regla de la bebida caliente?

Problema	No se puede entregar una bebida caliente a un socio dentro de una cancha de pádel o de una mesa de tenis de mesa techada. La regla toca el componente comercial (bebidas) y el deportivo (instalaciones).
Alternativas	A) En la clase Producto o Bebida, que consulte las reservas. B) En ReservaInstalacion, que rechace las ventas. C) En una clase propia, ReglaBebidaCaliente, que consulta la instalación del socio y evalúa la orden; y que cada Instalacion responda si restringe bebidas calientes.
Elegida	C

Ganamos	Los componentes comercial y deportivo no se mezclan: Bebida y ReservaInstalacion no conocen la regla. Al responder cada instalación por sí misma, la regla no pregunta de qué tipo es, y si se agrega otra instalación restringida solo se implementa ese método.
Sacrificamos	Una clase más y una dependencia del componente comercial hacia el deportivo, que es la única que existe entre ambos. La venta de una bebida pasa por un paso de verificación adicional.
Relacionada con	RF11; restricción de bebidas calientes. Decisión 2 de la sección 4.

D6. ¿Dónde está el control de la aplicación?

Problema	Hay que decidir quién recibe lo que pide el usuario, quién decide y a quién delega, para que la consola no tenga reglas del negocio y no haya una clase que haga todo.
Alternativas	A) Control centralizado: una clase gestora con todas las reglas. B) Control delegado: ConsolaMenu solo muestra y pide datos, CoordinadorAplicacion reparte el trabajo y cada decisión la toma su clase de verificación o de servicio. C) Control disperso: que cada clase de datos decida lo suyo.
Elegida	B
Ganamos	Los cambios quedan localizados: una regla nueva toca una clase de verificación, y una interfaz gráfica reemplazaría solo a ConsolaMenu. El coordinador es simple y no se vuelve una clase gestora.
Sacrificamos	Para seguir una funcionalidad completa hay que recorrer varias clases, y por eso se hicieron los diagramas de secuencia. Además, el coordinador depende de muchas clases, que es el precio de repartir el trabajo desde un solo punto.
Relacionada con	RNF5 y RNF1; estereotipos Coordinator, Controller e Interfacer de la sección 4. Estilo de control descrito en la sección 5.

D7. ¿Cómo se comunica la consola con la lógica?

Problema	En el Proyecto 1 CoordinadorAplicacion conocía todas las clases del sistema. Ahora hay tres consolas y un componente nuevo, y si cada consola llama directo a las clases de torneos, cualquier cambio en la lógica obliga a cambiar las tres consolas.

Alternativas	A) Que cada consola cree y use directamente GestorTorneos, Autenticador y AlmacenadorArchivos. B) Una interfaz FachadaTorneos con las operaciones que necesita la consola, implementada por una sola clase que reparte el trabajo. C) Mantener CoordinadorAplicacion y que las tres consolas lo usen.
Elegida	B.
Ganamos	Las consolas no conocen las clases internas de torneos. La interfaz sirvió como acuerdo entre las dos integrantes para trabajar al mismo tiempo, y se puede probar la lógica sin consola.
Sacrificamos	Una clase más y métodos que solo pasan la petición a otra clase. Si se agrega una operación nueva hay que agregarla en la interfaz y en su implementación.
Relacionada con	RNF5, RNF8; RF33 a RF42.

D8. ¿Una consola o tres?

Problema	El enunciado pide tres aplicaciones, una por tipo de usuario, pero las tres necesitan iniciar sesión, mostrar un menú, validar entradas y guardar al salir.
Alternativas	A) Tres clases independientes, cada una con todo su código. B) Una clase abstracta ConsolaBase con lo común y tres subclases con su menú y su main. C) Una sola consola que muestra un menú distinto según el rol.
Elegida	B.
Ganamos	El inicio de sesión, el manejo de errores y el guardado se escriben una sola vez. Cada consola solo define su menú y qué roles pueden entrar.
Sacrificamos	Hay que entender la clase base para modificar una consola, y un cambio en ConsolaBase afecta a las tres.
Relacionada con	RNF6, RNF7.

D9. ¿Dónde se decide cómo se cobra y cómo se premia un torneo?

Problema	Los torneos recreativos y competitivos tienen casi todas las reglas iguales, pero cobran y premian de forma distinta. Hay que decidir si eso se maneja con condiciones o con clases.

Alternativas	A) Una sola clase Torneo con un atributo tipo y condicionales en cada método. B) Torneo abstracta con calcularCobro y repartirPremios abstractos, y una subclase por tipo de torneo.
Elegida	B.
Ganamos	Agregar otro tipo de torneo no obliga a modificar Torneo ni la tabla; en GestorTorneos solo se agrega la opción al crearlo. Es la misma idea del RNF1 para los deportes.
Sacrificamos	Más clases, y para guardar y cargar los torneos hay que saber de qué subclase es cada uno.
Relacionada con	RNF1; RF33, RF40.

9. Cambios respecto al Proyecto 1

En esta tabla se resumen los cambios del diseño frente a la entrega final del Proyecto 1 y la razón de cada uno.

Parte	**Proyecto 1**	**Proyecto 2**	**Por qué**
Interfaz	Un solo programa (Principal) con CoordinadorAplicacion y ConsolaMenu.	Tres programas: ConsolaAdministrador, ConsolaParticipante y ConsolaEmpleado, con ConsolaBase y LectorEntradas.	El enunciado pide una aplicación por tipo de usuario y validar todas las entradas.
Comunicación consola - lógica	CoordinadorAplicacion llamaba directamente a todas las clases.	Interfaz FachadaTorneos implementada por ClubFachada.	Separar la consola de la lógica y poder dividir el trabajo.
Operaciones de comercio	Estaban dentro de CoordinadorAplicacion mezcladas con lecturas por teclado.	Pasaron a OperacionesComercio, sin leer ni imprimir.	Que se puedan usar desde varias consolas y probar con JUnit.
Errores	Se imprimían mensajes desde la lógica o se devolvía un String.	La lógica lanza OperacionRechazadaException y la consola muestra el mensaje.	Que el programa no se cierre y que la lógica no imprima nada.
Componentes	Cuatro componentes.	Siete: se agregaron Torneos y Fachada y se	Los torneos tienen reglas propias y la

		separó la consola de la autenticación.	consola creció mucho.
Usuarios	Administrador, Socio y Empleado (Entrenador y Fisioterapeuta).	Se agregan Estudiante y Externo. Usuario guarda los bonos.	Ahora pueden participar personas que no son socios.
Persistencia	AlmacenadorArchivos , y los datos se cargaban en Principal.	Se agrega PersistenciaClub. Los datos se cargan al iniciar y se guardan al salir de cualquier consola.	Lo pide el enunciado y así no se pierden datos entre programas.
Descuentos	Puntos y códigos de empleado.	Se agrega el bono de torneo, que no se acumula con los otros descuentos.	Premio de los torneos recreativos.
Pruebas	Programas de prueba Pruebas 1 a 12 que se revisaban en consola.	Pruebas con JUnit en la carpeta test.	Lo pide el enunciado.
Clases que se eliminan		CoordinadorAplicacion, ConsolaMenu y Principal.	Sus responsabilidades quedaron en las consolas nuevas y en OperacionesComercio.

10. Interfaz de consola

El sistema tiene tres programas que se ejecutan por separado, cada uno con su propio main dentro del paquete consola. Los tres trabajan sobre los mismos archivos, así que lo que hace un usuario en un programa lo ve otro usuario cuando abre el suyo.

10.1 Flujo de cada programa

1. Al abrir cualquiera de los programas se cargan los datos de la carpeta datos/.
2. Se pide el login y la contraseña. Hay tres intentos y si el rol del usuario no corresponde a esa consola no se deja entrar (por ejemplo, un socio no puede entrar a la consola de administrador).
3. Se muestra el menú en ciclo hasta que el usuario escoja la opción de salir.
4. Al salir, o si pasa un error inesperado, se guardan los datos en los mismos archivos.

10.2 Menús

Consola	Usuarios	Opciones del menú
Administrador	Administrador	1. Crear torneo 2. Iniciar torneo 3. Registrar resultado de un encuentro 4. Finalizar torneo y entregar premios 5. Consultar torneos 6. Reporte de ventas 7. Liberar cupos preferenciales 0. Salir
Participante	Socio, estudiante y externo	Antes de iniciar sesión pregunta si quiere crear una cuenta (estudiante o externo). 1. Inscribirse a un torneo 2. Salirse de un torneo 3. Ver mis torneos 4. Ver mis bonos 5. Consultar torneos 6. Comprar en la tienda 7. Comprar en la cafetería 0. Salir
Empleado	Entrenador y fisioterapeuta	1. Inscribirse a un torneo individual o de dobles 2. Salirse de un torneo 3. Ver mis torneos 4. Consultar torneos 5. Solicitar cambio de turno 0. Salir

10.3 Validación de entradas

Toda la lectura pasa por LectorEntradas, que es lo que cumple el RNF6. Sus métodos repiten la pregunta hasta que la respuesta sea válida:

- leerTextoNoVacio: no acepta campos vacíos.
- leerEntero y leerDecimal: no aceptan letras ni números fuera del rango permitido (por ejemplo una opción del menú que no existe o una tarifa negativa).
- leerFecha: solo acepta fechas con el formato AAAA-MM-DD.
- leerOpcion: muestra una lista numerada (por ejemplo los tipos de torneo o los torneos abiertos) y el usuario escoge el número, así no tiene que escribir el nombre exacto.
- leerSiNo: solo acepta s o n.

Las validaciones de formato las hace la consola, pero las reglas del negocio (si quedan cupos, si el usuario cumple la categoría, si el empleado tiene turno ese día, si el bono ya se usó) las decide la lógica. Cuando una regla no se cumple, la lógica lanza OperacionRechazadaException y ConsolaBase muestra el mensaje y vuelve al menú. De esta forma ninguna entrada hace que el programa se cierre.

En la tienda el participante puede usar un bono o un código de empleado, pero no los dos. Si escoge un bono, primero se revisa que haya existencias del producto y después se canjea, para que el bono no se gaste en una compra que no se pudo hacer.

10.4 Persistencia

Los archivos están en la carpeta datos/ del proyecto y sus nombres son fijos, el usuario nunca escribe una ruta. Los archivos del Proyecto 1 (socios, productos, ventas, puntos, turnos) los sigue manejando AlmacenadorArchivos, y los nuevos (usuarios, matrículas, torneos, inscripciones, encuentros, premios y bonos) los maneja PersistenciaClub. Si un archivo no existe, el programa arranca sin esos datos y lo crea al guardar.

11. Lógica de los torneos

La lógica de torneos está en el paquete deportivo y sigue el diagrama de clases de la sección 7.6. En esta sección se explica cómo quedó implementada cada regla y cómo se probó.

11.1 Implementación

Creación y cupos. GestorTorneos crea los torneos con un consecutivo (T1, T2, ...) y, según el tipo que llega de la consola, crea un TorneoRecreativo o un TorneoCompetitivo. No deja crear torneos con fecha anterior a hoy, y TorneoCompetitivo no acepta tarifa 0. Al crearse, Torneo calcula los cupos preferenciales con el 20% redondeado hacia arriba (en dobles sobre el número de parejas). Los cupos libres no se guardan en contadores: cuposRegularesLibres() y cuposPreferencialesLibres() se calculan recorriendo las inscripciones. Así, al desinscribir a alguien se libera automáticamente el mismo tipo de cupo que tenía. Con la HU-T14 el administrador puede convertir en regulares los preferenciales que nadie ha ocupado.
Inscripción. ClubFachada arma el participante (Jugador, Pareja o EquipoTorneo), revisa con GestorTorneos que ningún integrante empleado tenga turno el día de la semana del torneo y le pide al torneo el cobro con calcularCobro (0 en un recreativo o si todos los integrantes son empleados). Si hay cobro y no llega la confirmación del pago, la inscripción se rechaza. Después GestorTorneos decide si el participante va por cupo preferencial: junta el ELO de todos los que han jugado torneos de esa disciplina y el participante es preferencial si su ELO (el promedio, si es pareja o equipo) está entre los topK mejores (topK = 5). Quien no tiene historial tiene ELO 0 y va por cupo regular. Por último, Torneo.inscribir hace las validaciones antes de guardar nada: que las inscripciones estén abiertas, que ningún integrante ya esté inscrito, que cada integrante cumpla la categoría y que haya cupo. Si pidió preferencial y ya no quedan, pasa a regular; si tampoco quedan regulares, se rechaza.
ELO. Se reutilizó la clase Ranking del Proyecto 1. Cada usuario tiene un Ranking por disciplina que empieza en 1000 la primera vez que juega, y Torneo lo actualiza con la fórmula ELO (K = 32) cada vez que se registra un resultado con ganador.
Calendario y resultados. Al iniciar el torneo, GeneradorCalendario arma la liga todos contra todos con el método de rotación: el primer participante queda fijo y los demás rotan una posición por ronda. Si el número de participantes es impar se agrega un descanso en cada ronda. TablaTorneo da 3 puntos por victoria, 1 por empate (solo en deportes de conjunto) y 0

por derrota, y ordena con el desempate de la HU-T09: puntos, diferencia de marcador, marcador a favor y orden de inscripción.
Premios. finalizar() revisa que no queden encuentros pendientes y que los premios no se hayan entregado antes. TorneoRecreativo crea un BonoDescuento del 20% para cada integrante del ganador y se lo agrega a su usuario. TorneoCompetitivo reparte el fondo 60%, 30% y 10%, saltándose a los participantes que incluyen un empleado; si hay menos de tres premiados, lo que sobra queda para el club. BonoDescuento.canjear revisa que quien lo usa sea el titular, que no se haya usado y que la compra no tenga otro descuento.
Persistencia de los torneos. No se guardan los cupos, el calendario ni la tabla. Al cargar, PersistenciaClub vuelve a inscribir a los participantes en el mismo orden, inicia los torneos que ya habían empezado y registra otra vez los resultados, así los cupos, la tabla, el ELO y los premios quedan iguales a como estaban. En bonos.txt solo se guarda si cada bono ya se usó.

11.2 Pruebas con JUnit

Las pruebas están en la carpeta test del proyecto y son 55 en total: TorneoTest (32), TablaTorneoTest (4), BonoDescuentoTest (4), GestorTorneosTest (6) y ClubFachadaTest (9). Las de ClubFachadaTest son de integración, porque pasan por la fachada igual que la consola. Todas pasan. Esta es la relación entre historias de usuario y pruebas:

- HU-T01: testCuposPreferencialesIndividual, testCuposPreferencialesDobles, testDoblesConCuposImparesSeRechaza, testCompetitivoSinTarifaSeRechaza, testCrearTorneoAsignaIdsConsecutivos, testNoSeCreaTorneoConFechaPasada, testSoloElAdministradorCreaTorneos.
- HU-T02: testInscripcionOcupaCupoRegularYSumaAlFondo, testNoSePuedeInscribirDosVeces, testCategoriaPorNivel, testCategoriaPorEdad, testParejaOcupaDosCupos, testCompetitivoSinConfirmacionDePagoSeRechaza, testModalidadEquivocadaSeRechaza, testEquipoSoloDeSocios.
- HU-T03: testRegularesAgotadosAunqueQuedenPreferenciales, testPreferencialSinCupoPasaARegular, testSinHistorialNoEsPreferencial, testGanadorDeUnTorneoAnteriorEsPreferencial.
- HU-T04: testNoIniciaConMenosDeDos, testNoIniciaAntesDeLaFecha.
- HU-T05: testDesinscribirLiberaCupoYDevuelveTarifa, testDesinscribirPreferencialLiberaPreferencial, testNoSePuedeDesinscribirDespuesDeIniciar, testSoloElCapitanRetiraAlEquipo.
- HU-T06: testEmpleadoConTurnoEseDiaNoEstaLibre, testEmpleadoNoPagaEnCompetitivo.
- HU-T07: testRegistroDeEstudianteMatriculado, testRegistroDeEstudianteNoMatriculadoSeRechaza, testNoSeRepiteLoginNiDocumento, testAutenticarDevuelveElRol.
- HU-T08: testEmpateNoPermitidoEnIndividual, testEmpatePermitidoEnConjunto, testNoSeRegistraDosVecesElMismoEncuentro.
- HU-T09: testNoSeFinalizaConEncuentrosPendientes, testGanarDaTresPuntos, testDesempatePorDiferenciaYMarcadorAFavor, testSiTodoEsIgualGanaElQueSeInscribioPrimero.

- HU-T10: testRecreativoEntregaBonoAlGanador, testParejaGanadoraRecibeUnBonoCadaUno, testNoSeEntreganPremiosDosVeces, testCompetitivoReparte60_30_10SinEmpleados, testCompetitivoConDosParticipantesNoAsignaTercerPuesto, testCompetitivoSoloConEmpleadosNoTieneFondo.
- HU-T11: testCanjeDescuentaVeintePorCiento, testBonoUsadoSeRechaza, testNoSeAcumulaConOtroDescuento, testSoloLoUsaElTitular, testFlujoCompletoConBono.
- HU-T12: testFlujoCompletoConBono (consulta el detalle del torneo finalizado).
- HU-T13: testIniciarGeneraTodosContraTodos, testCalendarioConNumeroImparTieneDescansos.
- HU-T14: testLiberarPreferencialesLosVuelveRegulares, testNoSeLiberaConElTorneoEnCurso.

Además de las pruebas, se probaron las tres consolas a mano con los usuarios de datos/usuarios.txt: crear torneos, inscribirse, iniciar, registrar resultados, finalizar, canjear un bono en la tienda, y cerrar y volver a abrir el programa para revisar que los datos se guardaran.

11.3 Diferencias con el diagrama de clases

Al implementar se agregaron algunas cosas que no están en el diagrama:

- Usuario tiene un mapa de Ranking por disciplina (darElo y darRanking), que es de donde sale el ELO de promedioElo en ParticipanteTorneo.
- Torneo tiene liberarCuposPreferenciales() y FachadaTorneos tiene liberarCuposPreferenciales(admin, idTorneo) para la HU-T14. En la consola del administrador es la opción 7.
- ClubFachada guarda también la referencia a OperacionesComercio para cargar y guardar la parte comercial.
- Hay métodos de consulta pequeños (buscarInscripcion, darEncuentrosPendientes, darParticipantes y darPosiciones en Torneo) que no se pusieron en el diagrama para no llenarlo.

Riesgos y consistencia

1. **Persistencia y manejo de errores**

- Si un archivo de la carpeta datos/ no existe, el programa arranca sin esos datos y lo crea al guardar. Si un archivo no se puede leer, la consola muestra el error y el programa sigue funcionando. En una ejecución normal los datos se leen al iniciar cualquiera de los tres main y se guardan al salir.
- Según el RNF2, se implementa estrictamente la estrategia de validar todo primero y escribir después. Las clases de gestión y verificación comprueban todas las reglas de negocio antes de registrar o modificar el estado de los datos.
- Según el RNF3, la lectura y escritura de los archivos está exclusivamente encargada de la persistencia. Ninguna otra clase del modelo o de la interfaz accede directamente al disco.
- Ante cualquier intento que viole alguna regla de negocio, el sistema rechaza la operación de manera controlada y explica el motivo por el cual falló, permitiendo que la ejecución del programa continúe con normalidad.

2. **Cambios previstos**

Cambios previstos	Clases que se modificaron	Clases que no se modificaron
Agregar un deporte nuevo (RNF1)	Deporte, y las subclases de estadísticas y reglas de puntuación.	Almacenador archivos, autenticador y las vistas de consola.
Cambiar el valor del IVA (RNF4)	Parámetro financiero, clases de cálculo de impuestos y gestión de puntos.	Clases de dominio deportivo, gestión de instalaciones, control de turnos de personal.
Cambiar el formato de los archivos (RNF3)	Únicamente la clase de persistencia, Almacenador archivos.	Todo el modelo (socio, equipo, instalación, reglas de negocio).
Cambiar la consola por una interfaz gráfica	Consolas de prueba, parte visual.	Toda la lógica de negocio, validaciones y clases centrales del modelo.
Agregar un tipo de torneo nuevo (RNF1)	Una subclase nueva de Torneo con sus métodos calcularCobro y repartirPremios, y la opción nueva en GestorTorneos.crearTorneo.	Torneo, TablaTorneo, GeneradorCalendario, la fachada y las consolas.

3. **Coherencia entre diagramas y código**

Elemento	UML	Justificación
Métodos de verificación que retornan motivos de rechazo detallados en lugar de un boolean.	No	Se ajustó el código para cumplir con los requerimientos de salida y las pruebas.
Atributos de puntos en los socios y transacciones.	Si	Contemplado dentro de las clases del modelo.
Rol del entrenador como participante en partidos de práctica.	No	Se implementó como una regla de validación en la lógica del partido.

4. **Coherencia interna**

Cada componente y clase del sistema tiene una cohesión alta orientada a un solo dominio de responsabilidad.

- El paquete deportivo agrupa las disciplinas, instalaciones, partidos y rankings.
- La autenticación centraliza los usuarios y roles.
- El módulo de personal administra turnos y coberturas laborales. Esta separación garantiza que las clases trabajen exclusivamente en su contexto asignado, evitando acoplamientos.

Anexo: uso de inteligencia artificial generativa (IAG)

En este proyecto se usaron herramientas de IA generativa. Aquí se explica cuáles, para qué partes y cómo se revisó lo que produjeron.

Herramientas

Claude (Anthropic), en el chat de claude.ai, y Gemini (Google).

Uso por parte del proyecto

- Historias de usuario: las escribió Sara a partir del enunciado. La IA se usó para revisar la redacción de los criterios de aceptación y el formato Dado-Cuando-Entonces.
- Diagramas y documento de diseño: los diagramas de contexto, componentes, clases y secuencia los hizo Roxana en draw.io. Claude se usó para revisar que los diagramas fueran consistentes con los requerimientos y con el diseño del Proyecto 1, y para mejorar la redacción de algunas secciones del documento.
- Consolas, fachada y persistencia: Claude se usó para escribir y revisar el código de las tres consolas, LectorEntradas, la interfaz FachadaTorneos y la persistencia, a partir de lo que se había definido en el diseño.
- Lógica de torneos y pruebas: Sara hizo una primera versión de la lógica y de las pruebas JUnit con ayuda de Claude y Gemini. Al corregirla con Gemini las pruebas dejaron de pasar, y el código tampoco coincidía con el diagrama de clases. La versión final de la lógica de torneos (las clases del paquete deportivo de la sección 7.6), ClubFachada, la persistencia de los torneos y las 55 pruebas JUnit se escribieron con ayuda de Claude el 13 de noviembre, tomando como base el diagrama de clases y las historias de usuario del grupo. Con Claude también se redactó la sección 11 de este documento.

Cómo se revisó lo generado

- Se corrieron todas las pruebas JUnit y se verificó que pasaran.
- Se probaron las tres consolas a mano con los usuarios de prueba, incluyendo casos de error (pago sin confirmar, empleado con turno, código estudiantil que no está matriculado, empates no permitidos).
- Se comparó el código con el diagrama de clases y se anotaron las diferencias en la sección 11.3.
- Se revisó que el código y los comentarios correspondieran a lo visto en el curso.

Responsabilidad

Las decisiones de diseño, el reparto de responsabilidades entre clases y las reglas del club que se implementaron son del grupo. Las integrantes revisaron el código y el documento antes de entregarlos y responden por su contenido.